

Σavante

**Administración de servidores
de correo electrónico, sus
protocolos. Administración de
contenedores y microservicios**

ÍNDICE

1. Administración de servidores de correo electrónico	6
1.1. Correo electrónico	6
1.2. Elementos de un correo electrónico	7
1.2.1. Cliente de correo electrónico (MUA)	7
1.2.2. Mensajes de correo	9
1.2.3. Direcciones	10
1.2.4. Cuentas de correo	11
1.2.5. El servidor DNS en el envío de correo	11
1.3. Envío de un correo electrónico	12
2. Protocolos de correo electrónico	16
2.1. SMTP	16
2.1.1. Historia del protocolo SMTP	18
2.1.2. Cómo es una comunicación SMTP	18
2.1.3. Internacionalización	21
2.1.4. Correo saliente con SMTP	21
2.1.5. Diálogo entre un cliente SMTP y un servidor SMTP	23
2.1.6. Conexión del cliente SMTP al servidor SMTP	24
2.1.7. Comandos SMTP	24
2.2. POP	33
2.2.1. Operación básica	34
2.2.2. Estado de "Autorización"	35
2.2.3. Estado de "Transacción"	36
2.2.4. Estado de "Actualización"	38
2.2.5. Comandos que brindan seguridad	38
2.2.6. Inseguridades del protocolo POP	40
2.3. IMAP	41
2.4. MIME	41
2.5. S/MIME	42

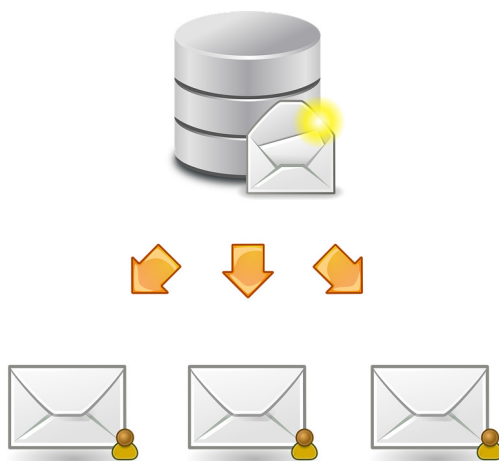
2.6. Otros protocolos	42
2.6.1. RFC 2142	42
2.6.2. Justificación y Alcance	43
2.6.3. Invariantes	45
2.6.4. Nombres de buzones	45
2.6.4.1. Nombres de Buzones Relacionados con la Empresa	45
2.6.4.2. Nombres de Correos de Red	46
2.6.4.3. Nombres de Correos de Apoyo para Servicios de Internet Específicos	46
2.6.4.4. Lista de Correo: Buzón de Administración	47
2.6.5. Administración del Buzón de Nombres de Dominio	47
2.6.6. Correos de los Sistemas Autónomos	47
2.6.7. Consideraciones de Seguridad	48
3. Administración del correo electrónico	48
3.1. Monitorización	48
3.2. Seguridad	52
3.2.1. Entorno del servidor	52
3.2.2. Configuración segura de los servidores	53
3.2.3. Seguridad en servicios de correo	54
3.2.4. Auditorías del sistema	55
3.2.5. Seguridad en clientes de correo	56
3.2.5.1. Clientes de correo	56
3.2.5.2. Correo WEB	56
3.2.6. Dispositivos móviles	57
3.2.7. Gestión de contraseñas	57
4. Administración de contenedores y microservicios	59
4.1. Que son los microservicios	60
4.2. Ventajas de la arquitectura de microservicios	61
4.2.1. Especialización del equipo	61
4.2.2. Altamente escalable	61
4.2.3. Independencia y versatilidad, aplicaciones más abiertas	62

4.2.4. Consistencia: aislamiento de fallos y capacidad de recuperación	62
4.2.5. Rapidez de respuesta implementación y actualización	62
5. Arquitectura de microservicios frente a la arquitectura monolítica	63
6. El cambio a microservicios	64
6.1. Desafíos (problemas) en la arquitectura de microservicios	66
6.2. Implementación de una arquitectura de microservicios	68
6.3. Ejemplos de sistemas con arquitectura de microservicios	69
6.3.1. Netflix	69
6.3.2. Spotify	70
6.3.3. eBay	70
7. El uso de contenedores (Kubernetes y Docker)	71
7.1. Microservicios y contenedores	72
7.2. Orquestación de contenedores	73
7.3. Seguridad en los contenedores	74
7.4. Los contenedores en La Nube	76
8. Malla de servicios (Service Mesh)	76
8.1. Arquitectura	78
8.2. Características	80
8.2.1. Multiusuario	80
8.2.2. Seguridad	81
8.2.3. Observabilidad y análisis	82
8.2.4. Cumplimiento de políticas y reglas	84
8.2.5. Control del tráfico	85
8.2.6. Resiliencia	87
8.3. Consideraciones del diseño	88
8.4. Pruebas	90
8.5. Ejemplos de malla de servicios	91

9. Soluciones de administración de contenedores	92
9.1. Docker	92
9.1.1. Comandos principales	96
9.2. Azure Kubernetes Service	101
9.3. Kubernetes	101
9.3.1. Objetos básicos de Kubernetes	102
9.3.2. Objetos avanzados	103
9.3.3. Principales protocolos utilizados en Kubernetes	103
9.4. Hyper-V Containers	105
9.5. OpenShift	105
9.6. Otras soluciones de Orquestación de Contenedores	105
10. Bibliografía	106



1. Administración de servidores de correo electrónico



Fuente: (<https://pixabay.com/es/correo-electr%C3%B3nico-comercializaci%C3%B3n-156765/>)

La persona que se encarga de la administración de los servidores de correo electrónico es el administrador de servidor de correo o, en su defecto, el administrador de sistemas.

Vamos a ver qué es un correo electrónico, sus elementos y cómo funciona un servidor de correo antes de ver la forma de administrarlo.

1.1. Correo electrónico



Fuente:
Correo_electrónico_redireccionado de Wikipedia

Un correo electrónico es un servicio de red que permite a los usuarios enviar y recibir mensajes (también denominados mensajes electrónicos o cartas digitales) mediante redes de comunicación electrónica.



El nombre "correo electrónico", en inglés es: electronic mail, por lo que comúnmente, al abreviarlo se le denomina e-mail o email.

El término «correo electrónico» proviene de la analogía con el correo postal: ambos sirven para enviar y recibir mensajes, y se utilizan «buzones» intermedios (servidores de correo).

Por medio del correo electrónico se puede enviar texto, y todo tipo de archivos digitales, aunque existen un límite (en cuanto al tamaño) en los archivos adjuntos.

Los sistemas de correo electrónico se basan en un modelo de almacenamiento y reenvío, de modo que no es necesario que ambos extremos se encuentren conectados simultáneamente.

Para ello se emplea un servidor de correo que hace las funciones de intermediario, guardando temporalmente los mensajes antes de enviarse a sus destinatarios.

Para poder enviar o recibir mensajes de un correo electrónico es necesario disponer de una cuenta de correo (existen multitud de servidores de correo, gratuitos y de pago, para poder crear una cuenta de correo).

1.2. Elementos de un correo electrónico

Vamos a ir desarrollando los diferentes elementos para el funcionamiento de envío y recepción de un correo electrónico, que son:

- Cliente de correo electrónico (MUA).
- Mensajes de correo.
- Direcciones.
- Cuentas.
- Servidor DNS.

1.2.1. Cliente de correo electrónico (MUA)

Un cliente de correo electrónico (MUA) es un programa de ordenador usado para leer y enviar mensajes de correo electrónico.

Los clientes de correo deben soportar protocolos como POP3 e IMAP para comunicarse con un MTA remoto localizado en la máquina de proveedores de correo electrónico.

IMAP está optimizado para almacenar correos electrónicos en el servidor.



POP3 asume generalmente que los mensajes de correo electrónico se descargan al cliente.

La gran mayoría de clientes de correo electrónico emplean el protocolo SMTP para enviar los mensajes de correo electrónico.

Existen también programas de correo electrónicos basados en la Web, denominados *webmail* o correo web.

Un importante estándar soportado por la mayoría de los clientes de correo electrónico es MIME.

MIME se emplea para el envío de archivos binarios adjuntos al correo.

Ejemplos de clientes de correo son:

- **De escritorio:**

- Microsoft Outlook.
- Mozilla Thunderbird.
- IncrediMail.
- Apple Mail.
- Opera Mail.

- **En Web:**

- SquirrelMail.
- Horde.
- OpenWebMail.
- RoundCube.



Clientes de correo web. Fuente: (https://commons.wikimedia.org/wiki/File:Men%C3%BA_webmail.jpg)



1.2.2. Mensajes de correo



Fuente:
<https://pixabay.com/es/en-correo-electr%C3%B3nico-enviar-1019990/>

La estructura de un mensaje de correo tiene tres elementos básicos:

- **Para** (Destinatario).

Una o varias direcciones (separadas por ;) de correo a las que se enviará el mensaje.

- **Asunto.**

Breve descripción del contenido del mensaje. Es opcional. Se puede ver antes de abrir el correo.

- **Mensaje.**

Texto con o sin formato, imágenes, etc.

También tenemos la opción de adjuntar archivos.

Además del campo Para existen los campos CC y CCO. Estos son opcionales y sirven para añadir destinatarios de forma especial:

- **CC (Copia de Carbón).**

Quienes estén en esta lista recibirán también el mensaje, pero verán que no va directamente dirigido a ellos.

- **Campo CCO (Copia de Carbón Oculta).**

Igual que CC, pero la lista de destinatarios aquí incluida no pueden verla ninguno de los destinatarios.



Este es el asunto del mensaje

Para

Cc

Cco

Este es el asunto del mensaje

Aquí viene el cuerpo del mensaje

Enviar



Guardado



Mensaje de correo electrónico

1.2.3. Direcciones

Una dirección de correo electrónico es una cadena de texto que identifica a una persona.

Cada dirección es única.

La estructura de una dirección de correo es: usuario@dominio

- **Usuario:** Normalmente lo podemos elegir (si está disponible en el proveedor).
- La @ es obligatoria.

El símbolo arroba forma parte de todos los correos electrónicos y está especificada en la norma RFC 5321.

- **Proveedor:** Proveedor del servicio de correo.



Atención

Las direcciones de correo electrónico no diferencian entre mayúsculas y minúsculas.

1.2.4. Cuentas de correo

Es un servicio online que provee un espacio para el almacenamiento de mensajes de correo electrónico.

Una cuenta se asocia a un único usuario, el cual puede acceder a su cuenta a través de un nombre de usuario y contraseña.

Este servicio lo ofrecen los servidores de mail.

1.2.5. El servidor DNS en el envío de correo

El DNS (Domain Name System o sistema de nombres de dominio) es una tecnología esencial para el funcionamiento del correo electrónico, ya que permite resolver la dirección IP de los servidores de correo asociados a un dominio. En particular, cuando se envía un mensaje a través de Internet, el servidor emisor (MTA, Mail Transfer Agent) consulta el DNS para obtener el registro MX (Mail Exchange) del dominio de destino, es decir, la parte que está a la derecha de la @ en la dirección del destinatario.

El registro MX devuelve una lista de servidores que aceptan correo entrante para ese dominio, junto con un valor de prioridad. Esto permite definir múltiples servidores de correo, organizados jerárquicamente según su preferencia de uso. De este modo, si el servidor principal no está disponible, el emisor intentará establecer la conexión SMTP con el siguiente servidor en la lista, asegurando tolerancia a fallos.

Este mecanismo de resolución es imprescindible para la correcta entrega del correo electrónico y forma parte de los estándares de funcionamiento de Internet. Además, su comportamiento puede comprobarse con herramientas de diagnóstico como nslookup, desde la línea de comandos. Por ejemplo, en Windows, el comando `nslookup -type=MX midominio.com` permite consultar el registro MX de un dominio determinado y verificar qué servidores están autorizados para recibir mensajes en su nombre.



Ejemplo

Puedes hacer la comprobación desde Windows con el comando "nslookup" desde "cmd".

Ejecuta "nslookup 81.89.32.200 10.31.15.6" y te dará el nombre del servidor que corresponde a la primera dirección IP, tras realizar la consulta en el servidor que has indicado en la segunda dirección IP.

El protocolo de aplicación DNS (de la capa 7 del modelo OSI) se explica más ampliamente en la Ud. 8 del bloque IV, "Internet: arquitectura de red." en el epígrafe 2.3. DNS.

1.3. Envío de un correo electrónico



Fuente: e-mail-97624_960_720 de Pixabay

Cuando se envía un correo electrónico, el mensaje se enruta, (va pasando) de servidor a servidor hasta llegar al servidor de correo electrónico del receptor.

De forma más detallada, se siguen los siguientes pasos:

- El mensaje se envía al servidor del correo saliente (MTA, Mail Transport Agent o Agente de Entrega de Correo).
- La labor del MTA es transportar el mensaje al MTA del destinatario.
- En Internet, los MTA se comunican entre sí usando el protocolo SMTP (por lo que a veces nos referimos a ellos como servidores SMTP).



- Una vez que el MTA del destinatario recibe el correo electrónico, lo entrega al servidor de correo entrante (MDA, Mail Delivery Agent o Agente de Entrega de Correo).
- El MDA almacena el correo electrónico hasta que el usuario lo acepte.
- El MUA proporciona al usuario acceso al correo una vez que el MDA lo ha almacenado en el servidor de correo entrante.

Puede hacerlo a través de 2 protocolos:

- **POP3.**

Post Office Protocol o Protocolo de Oficina de Correo.

Es el más antiguo.

Elimina por defecto los mensajes del servidor después de descargarlos al cliente a menos que se realice una configuración específica en este último para conservar una copia en el servidor.

A los servidores que usan este protocolo se les suele llamar servidores POP.

- **IMAP.**

Internet Message Access Protocol o Protocolo de Acceso a Mensajes de Internet).

Se utiliza para coordinar el estado de los correos (leído, eliminado, etc.) en múltiples clientes de correo.

Con IMAP, se guarda una copia de cada mensaje en el servidor, de manera que esta tarea de sincronización se pueda realizar.

A los servidores que usan este protocolo se les suele llamar servidores IMAP.



Ejemplo

Hagamos una analogía con el servicio de correos.

Los MTA serían las oficinas de correos (clasifican y transportan (transmiten) los mensajes).



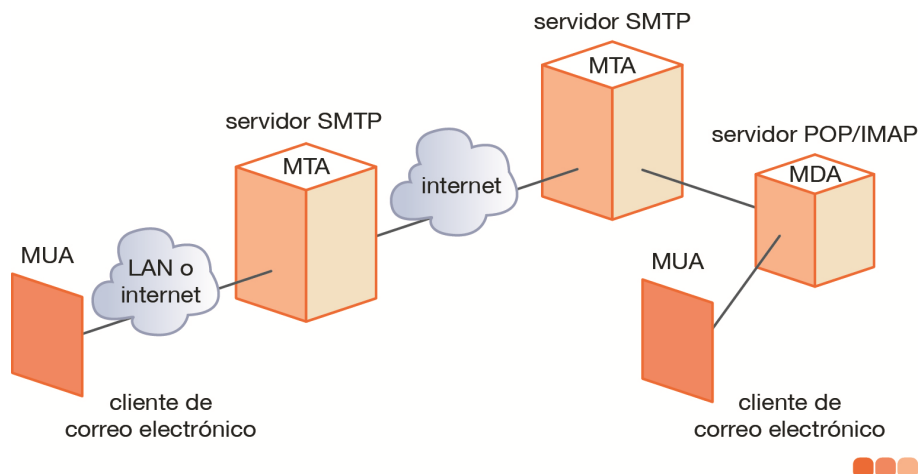
Los MDA serían los buzones donde se almacenan los mensajes (tienen una determinada capacidad y algunos paquetes podrían no caber).

Los destinatarios lo recogen cuando quieren (no necesitan estar en el buzón (conectados) para poder recibirlos).

Para acceder al MDA, se utiliza un sistema de usuario/contraseña para garantizar que el mensaje no pueda leerlo cualquiera.

La recuperación del correo se logra a través de un programa de *software* llamado MUA (Mail User Agent o Agente Usuario de Correo). Normalmente se le conoce por cliente de correo electrónico.

A continuación, te mostramos una imagen para que lo veas más claro.



Elementos implicados en el envío de un correo electrónico

Algunos MTA son:

- **Exim Internet Mailer:**

Exim es un agente de transferencia de mensajes (MTA) desarrollado en la **Universidad de Cambridge** para su uso en sistemas **Unix** conectados a Internet.

Está disponible gratuitamente bajo los términos de la **Licencia Pública General de GNU**.



- **POSTFIX:**

Es el servidor de correo LINUX de **Wietse Venema** que comenzó su vida en la investigación de **IBM** como una alternativa al ampliamente utilizado programa **Sendmail**. Ahora en **Google**, **Wietse** continúa apoyando **Postfix**.

- **Microsoft Exchange Server:**

Es un servidor de correo electrónico al que se puede acceder mediante un navegador web y es compatible con aplicaciones cliente como **Microsoft Outlook**.

Algunas de sus características son:

- **Exchange Server Deployment Assistant:**

Es una herramienta gratuita en línea que ayuda a implementar rápidamente Exchange en la organización al hacer algunas preguntas y crear una lista de verificación de implementación personalizada.

- **Arquitectura:**

Exchange utiliza una arquitectura de bloques de construcción única que proporciona servicios de correo electrónico para implementaciones de todos los tamaños.

- **Exchange Server permissions:**

Microsoft Exchange Server incluye un gran conjunto de permisos predefinidos, basados en el modelo de permisos de Control de acceso basado en roles (RBAC), que puede usar de inmediato para otorgar fácilmente permisos a sus administradores y usuarios.

- **Mail flow and the transport pipeline:**

En Exchange Server, el flujo de correo se produce a través de la canalización de transporte.

La **canalización de transporte** es una colección de servicios, conexiones, componentes y colas que trabajan juntos para enrutar todos los mensajes al categorizador en el servicio de transporte en un servidor de buzón de Exchange dentro de la organización.

- **Database availability groups:**

Un grupo de disponibilidad de base de datos (DAG) es el componente base del marco de alta disponibilidad y resistencia del sitio del servidor de buzón integrado en Microsoft Exchange Server.

Un DAG es un grupo de hasta 16 servidores de buzón de correo que aloja un conjunto de bases de datos y proporciona recuperación automática a nivel de base de datos de fallas que afectan a servidores o bases de datos individuales. Los posibles estados de una base de datos solo serán "activos" o "pasivos".



2. Protocolos de correo electrónico

Existen diferentes protocolos, que vas a estudiar a continuación, para la transmisión de correos electrónicos:

- SMTP.
- POP.
- IMAP.
- MIME.
- S/MIME.

2.1. SMTP

SMTP (Simple Mail Transfer Protocol o Protocolo Simple de Transferencia de Correo).

SMTP es un protocolo de la capa de aplicación.

Utiliza los siguientes puertos:

- Los puertos 25, 587 y 2525 para conexiones no encriptadas.
- Los puertos 465 y 25025 para las encriptadas.



+ Info

La capa de aplicación es la más alta del modelo OSI.

Los protocolos de esta capa se utilizan para intercambiar datos entre los programas que se ejecutan en los hosts de origen y destino.

En este tema no hablaremos del modelo OSI.



Características de SMTP:

- Es un protocolo de red basado en textos utilizados para el intercambio de mensajes de correo electrónico entre dispositivos.
- Está basado en el modelo cliente-servidor.
- La comunicación entre el cliente y el servidor consiste enteramente en líneas de texto compuestas por caracteres ASCII.
- El tamaño máximo permitido para estas líneas es de 1000 caracteres.
- Las respuestas del servidor constan de un código numérico de tres dígitos, seguido de un texto explicativo.
- El número va dirigido a un procesamiento automático de la respuesta, mientras que el texto permite que un humano interprete la respuesta.
- En el protocolo SMTP todas las órdenes, réplicas o datos son líneas de texto, delimitadas por <CRLF>.



+ Info

<CRLF> es la combinación de CR (retorno de carro) y LF (salto de línea).

Se utiliza como marcador de nueva línea.



#AclaraT

El puerto 587 no conlleva cifrado por defecto, el cifrado se negociará con el servidor. Si configuramos la extensión o comando STARTTLS, estaremos negociando con el servidor de correo una conexión cifrada que éste último podrá aceptar o no.



Por lo tanto, la transmisión de un correo electrónico con STARTTLS hasta que la negociación de encriptación con el servidor haya llegado a buen puerto (nunca mejor dicho XD) estará expuesta.

Los comandos SMTP HELO o EHLO (identificación del cliente ante el servidor), MAIL FROM (remitente) y RCPT TO (destinatario) así como los datos que les acompañan son visibles pues están en texto plano.

Tras el envío de estos mismos solicitaremos la conexión TLS con el comando STARTTLS todavía en texto plano.

Si la negociación acaba con éxito contenido del mensaje y credenciales de inicio de sesión serán encriptadas.

Así pues, desde un inicio el puerto 587 transmite usando SMTP protocolo sin cifrado, cambiará a una transmisión TLS (encriptada) con el comando STARTTLS si el servidor lo acepta.

Por otro lado, los puertos 465 y 25025 salen por defecto con la transmisión cifrada.

2.1.1. Historia del protocolo SMTP

Definido inicialmente en agosto de 1982 por el **RFC 821** (para la transferencia) y el **RFC 822** (para el mensaje).

Son estándares oficiales de Internet que fueron reemplazados respectivamente por el **RFC 2821** y el **RFC 2822**, que a su vez lo fueron por el **RFC 5321** y el **RFC 5322**.

A la fecha en que se actualiza este documento (mediados de 2020) el protocolo en vigor es el **RFC 5321**, por tanto, la información indicada a continuación, hace referencia a este protocolo.

2.1.2. Cómo es una comunicación SMTP

Según el protocolo RFC 5321 una transacción SMTP se compone de los comandos (los indicaremos un poco más adelante).

- MAIL FROM.
- RCPT TO.
- DATA.



El mensaje es enviado por el cliente después de que éste manda la orden DATA al servidor.

Ejemplo:

```
S: 220 Servidor SMTP
C: HELO miequipo.midominio.com
S: 250 Hello, please to meet you
C: MAIL FROM: <yo@midominio.com>
S: 250 Ok
C: RCPT TO: <destinatario@sudominio.com>
S: 250 Ok
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: Subject: Campo de asunto
C: From: yo@midominio.com
C: To: destinatario@sudominio.com
C:
C: Hola.
C: Esto es una prueba.
C: Hasta luego.
C:
C: .
C: <CR><LF>.<CR><LF>
S: 250 Ok: queued as 12345
C: quit
S: 221 Bye
```

El mensaje está compuesto por dos partes:

- **Cabecera:**

En el ejemplo las tres primeras líneas del mensaje son la cabecera. En ellas se usan unas palabras clave para definir los campos del mensaje. Estos campos ayudan a los clientes de correo a organizarlos y mostrarlos. Los más típicos son subject (asunto), from (emisor) y to (receptor). Estos dos últimos campos no hay que confundirlos con las órdenes MAIL FROM y RCPT TO, que pertenecen al protocolo, pero no al formato del mensaje.



- **Cuerpo del mensaje:**

Es el mensaje propiamente dicho. En el SMTP básico está compuesto únicamente por texto, y finalizado con una línea en la que el único carácter es un punto.

SMTP vs Recuperación de correo

El Protocolo Simple de Transferencia de Correo (SMTP) es el protocolo estándar para el envío de correos electrónicos entre servidores. Su función principal es entregar mensajes de un servidor de correo a otro, siguiendo una ruta determinada por el dominio del destinatario. Los servidores de correo utilizan colas para almacenar temporalmente los mensajes y enviarlos cuando las condiciones de red lo permiten, lo que facilita la entrega de correo incluso en conexiones intermitentes. SMTP opera antes de que un mensaje llegue a su destino final, manejando el envío y la entrega a través de servidores.

A diferencia del SMTP, los protocolos POP (Post Office Protocol) e IMAP (Internet Message Access Protocol) están diseñados para permitir a los usuarios acceder y gestionar su correo electrónico de forma individual. Concebidos para funcionar después de que los correos han sido entregados. Estos protocolos se encargan de la recuperación de mensajes, la organización de buzones y otras funciones relacionadas con la interacción del usuario con su correo, mientras que con el protocolo SMTP los mensajes salientes de varios usuarios pueden confluir en el servidor de correo saliente.

Inicio remoto de mensaje en cola

Es una característica de SMTP que permite a un host remoto para iniciar el procesamiento de la cola de correo en el servidor por lo que puede recibir mensajes destinados a ella mediante el envío del comando TURN.

Esta característica se considera insegura, pero usando el comando ETRN en la extensión RFC 1985 funciona de forma más segura.

Petición de Reenvío de Correo Bajo Demanda (ODMR)

On-Demand Mail Relay (ODMR por sus siglas en inglés) es una extensión de SMTP estandarizada en la RFC 2645 que permite que el correo electrónico sea transmitido al receptor después de que él ha sido aprobado.

Usa la orden de SMTP ampliada ATRN, disponible para las direcciones de IP dinámicas.

El cliente publica EHLO y órdenes de AUTH de servicios ODMR de correo.

ODMR comienza a actuar como un cliente SMTP y comienza a enviar todos los mensajes dirigidos a un cliente usando el protocolo SMTP, al iniciar sesión, el cortafuegos o el servidor pueden bloquear la sesión entrante debido a IP dinámicas.

Sólo el servidor ODMR, el proveedor del servicio, debe escuchar las sesiones SMTP en una dirección de IP fija.



2.1.3. Internacionalización

Los usuarios cuyo lenguaje base no es el latín, han tenido dificultades con el requisito de correo electrónico en América, para resolver este problema se creó RFC 6531, proporcionando características de internacionalización de SMTP; la extensión SMTPUTF8.

RFC 6531 proporciona soporte para caracteres de varios bytes y no para ASCII en las direcciones de correo electrónico.

El soporte de internacionalización actualmente es limitada, pero hay un gran interés en la ampliación del RFC 6531. (RFC en países como en China, que tiene una gran base de usuarios en América).

2.1.4. Correo saliente con SMTP

Un cliente de correo electrónico tiene que saber la dirección IP de su servidor SMTP inicial y esto tiene que ser dado como parte de su configuración (usualmente dada como un nombre DNS). Este servidor enviará mensajes salientes en nombre del usuario.

Restricción de acceso y salida al servidor de correo

En un ambiente de servidores, los administradores deben tomar medidas de control en donde los servidores estén disponibles para los clientes.

Esto permite implementar seguridad frente a posibles amenazas. Anteriormente, la mayoría de los sistemas imponían restricciones de uso de acuerdo a la ubicación del cliente, sólo estaba permitido su uso por aquellos clientes cuya dirección IP es una de las controladas por los administradores del servidor.

Los servidores SMTP modernos se caracterizan por ofrecer un sistema alternativo, el cual requiere de una autenticación mediante credenciales por parte de los clientes antes de permitir el acceso.

Para facilitar esta función existe el protocolo **SPF (Sender Policy Framework)** es una protección contra la falsificación de direcciones en el envío de correo electrónico.

Identifica, a través de los registros de nombres de dominio (DNS), a los servidores de correo SMTP autorizados para el transporte de los mensajes a un dominio determinado.

Este convenio busca ayudar para disminuir abusos como el spam y otros males del correo electrónico.

Restringir el acceso por ubicación

Mediante este sistema, el servidor SMTP relativo al ISP no permitirá el acceso de los usuarios que están fuera de la red del ISP.

Específicamente, el servidor solo puede permitir el acceso de aquellos usuarios cuya dirección IP fue proporcionada por el ISP, lo cual es equivalente a exigir que estén conectados a internet mediante el mismo ISP.



Un usuario móvil suele estar a menudo en una red distinta a la normal de su ISP, y luego descubrir que el envío de correo electrónico falla porque la elección del servidor SMTP configurado ya no es accesible.

Este sistema tiene distintas variaciones, por ejemplo, el servidor SMTP de la organización sólo puede proporcionar servicio a los usuarios en la misma red, esto se hace cumplir mediante cortafuegos para bloquear el acceso de los usuarios en general a través de Internet. O puede que el servicio realice comprobaciones de alcance en la dirección IP del cliente.

Estos métodos son utilizados normalmente por empresas e instituciones, como las universidades que proporcionan un servidor SMTP para el correo saliente solo para su uso interno dentro de la organización.

Sin embargo, la mayoría de estos organismos utilizan ahora métodos de autenticación de cliente:

- Al restringir el acceso a determinadas direcciones IP, los administradores de servidores pueden reconocer fácilmente la dirección IP de cualquier agresor.
- Como está representada una dirección significativa para ellos, los administradores pueden hacer frente a la máquina o usuario sospechoso.
- Cuando un usuario es móvil, y puede utilizar diferentes proveedores para conectarse a internet, este tipo de restricción de uso es costoso, y la alteración de la configuración perteneciente a la dirección de correo electrónico del servidor SMTP saliente resulta ser poco práctica.
- Es altamente deseable poder utilizar la información de configuración del cliente de correo electrónico que no necesita cambiar.

En entornos reales, uno de los servidores SMTP más utilizados para gestionar el correo saliente es Postfix, un agente de transporte de correo (MTA, por Mail Transport Agent) presente en la mayoría de sistemas Unix/Linux. Para implementar políticas como autenticación de clientes, control de acceso por IP o personalización del nombre del servidor en el saludo SMTP, es necesario configurar adecuadamente su archivo principal: `main.cf`.

Postfix

Postfix como decíamos, es ampliamente utilizado en sistemas Unix/Linux para el envío y la recepción de correos electrónicos. Su configuración principal se realiza a través del archivo `main.cf`, ubicado en `/etc/postfix/`, donde se definen las variables que controlan su comportamiento.

Para cambiar valores en Postfix, se edita directamente este archivo con un editor de texto (por ejemplo, nano o vi). Algunas variables comunes que pueden modificarse son:

- `myhostname`: define el nombre del servidor de correo (por ejemplo, `myhostname = mail.ejemplo.com`).
- `mydomain`: establece el dominio principal usado por el servidor (por ejemplo, `mydomain = ejemplo.com`).



- **relayhost:** permite enviar el correo a través de otro servidor SMTP (por ejemplo, `relayhost = [smtp.gmail.com]:587`).
- **mynetworks:** lista de direcciones IP o redes desde las cuales se permite el envío sin autenticación (por ejemplo, `mynetworks = 127.0.0.0/8, 192.168.1.0/24`).
- **smtp_helo_name:** especifica el nombre que el servidor enviará en los comandos HELO/EHLO al conectarse con otros servidores (por ejemplo, `smtp_helo_name = CORREOBIBLIOTECA`).

Una vez realizados los cambios, es imprescindible reiniciar el servicio para que surtan efecto. Esto puede hacerse con el comando `sudo systemctl restart postfix` o, en sistemas más antiguos, `sudo service postfix restart`. De este modo, la nueva configuración queda activada en el servidor de correo.

2.1.5. Diálogo entre un cliente SMTP y un servidor SMTP

El diálogo entre un cliente SMTP y un servidor SMTP se basa en un conjunto de comandos enviados por el cliente SMTP, que son palabras en formato texto ASCII legibles con facilidad y unos códigos de respuesta numéricos seguidos de un texto que explica dicho código, que son enviados por el servidor SMTP.

• Códigos de respuesta.

Los códigos de respuesta están formados por tres dígitos, cada uno de los cuales tiene un significado.

- El primero de los dígitos de un código de respuesta, indica si el comando funcionó correctamente o si falló. La siguiente tabla representa los significados del primer dígito de un código de respuesta (RFC 5321):

Primer dígito: Significado:

2: Respuesta de finalización positiva. El comando finalizó correctamente.

3: Respuesta positiva intermedia. Se aceptó el comando, pero se espera a que el cliente envíe más información.

4: Respuesta de finalización negativa temporal. El comando ha sido rechazado, pero el cliente debería intentarlo de nuevo.

5: Respuesta de finalización negativa permanente. Se rechazó el comando y no se debe reintentar sin corregir el error.

- El segundo de los dígitos de un código de respuesta especifica la categoría de la respuesta enviada. La siguiente tabla indica el significado del segundo dígito:

Segundo dígito: Significado.

0: Sintaxis.



1: Información.

2: Conexiones.

(3, 4 y 5 no están asignados en SMTP.)

- **El tercer dígito** de un código de respuesta especifica más el significado de las categorías de las respuestas.

2.1.6. Conexión del cliente SMTP al servidor SMTP

Vamos a ver cómo se realiza la conexión del cliente SMTP al servidor SMTP.

Para establecer una conexión, el cliente se conecta al servidor mediante el puerto TCP 25.

Cuando se establece la conexión, el cliente recibe un código de respuesta que le indica si el servidor está en disposición de aceptar la conexión y si es capaz de abrir una sesión o bien si el servicio de correo no está disponible en ese momento.

Los códigos de respuesta posibles son:

- **220:** El servicio de correo está disponible.
- **421:** El servicio de correo no está disponible.

2.1.7. Comandos SMTP

Vamos a indicar los comandos SMTP en detalle:

- **DATA:**

Este comando indica al servidor que el texto que va a continuación del comando es ya el mensaje de correo que debe de llevarse al destinatario indicado por el encabezado del mensaje.

El texto del mensaje debe de estar de acuerdo con el estándar del formato de mensaje de Internet, descrito en la RFC 822. Este texto del mensaje debe finalizar con un punto, que tiene que ir precedido (del comando anterior) y sucedido de los caracteres de retorno carro/avance de línea, #13#10.

El funcionamiento es sencillo, se envía el comando y el servidor debería responder con el código de respuesta 354. El paso siguiente es enviar el mensaje, al término del cual se debería de recibir el código de respuesta 250.



Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
- **354:** Comenzar la introducción del correo, acabando con <CRLF>.<CRLF>.
- **421:** El servicio no está disponible.
- **451:** Se abandonó la acción por un error de procesamiento local.
- **452:** No se produjo la acción por que el disco no tiene espacio de almacenamiento suficiente.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.
- **501:** Error en la sintaxis de los parámetros del comando.
- **503:** Secuencia de comandos incorrecta.
- **552:** Abandono de la acción porque se superó la reserva de espacio.
- **554:** Se produjo un fallo en la transacción.
- **EXPN - (expandir):**

Este comando se utiliza para verificar las listas de correo. Si se le pasa como parámetro un nombre de lista de correo, el servidor nos devuelve los nombres de usuario y las direcciones de los destinatarios de la lista de correo.

Sintaxis: `EXPT nombre_lista_de_correo#13#10`

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
- **421:** El servicio no está disponible.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.
- **501:** Error en la sintaxis de los parámetros del comando.
- **502:** El comando no está implementado.
- **504:** El parámetro del comando no está implementado.
- **550:** La acción no se realizó por que no se ha encontrado el buzón.



- **HELO:**

Este comando es el encargado de iniciar el dialogo SMTP.

Este comando tiene como parámetro el nombre del cliente para establecer su identidad.

El servidor responderá con un código de respuesta 250 seguido del nombre del servidor.

Sintaxis: `HELO nombre_cliente#13#10`

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
- **421:** El servicio no está disponible.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.
- **501:** Error en la sintaxis de los parámetros del comando.
- **504:** El parámetro del comando no está implementado.

- **HELP:**

Este comando hace que el servidor envíe información de ayuda sobre todos los comandos o sobre un comando en concreto.

Sintaxis: `HELP [ cadena-comandos ]#13#10`

Los posibles códigos de respuesta a este comando son:

- **211:** El sistema tiene disponible la ayuda.
- **214:** Mensaje de información de ayuda.
- **421:** El servicio no está disponible.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.
- **501:** Error en la sintaxis de los parámetros del comando.
- **502:** El comando no está implementado.
- **504:** El parámetro del comando no está implementado.



- **MAIL:**

Este comando indica al servidor el inicio de un mensaje de correo y le indica además quien es el remitente del mensaje.

Sintaxis: MAIL FROM: nombre_remitente@host_remitente#13#10

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
 - **421:** El servicio no está disponible.
 - **451:** Se abandonó la acción por un error de procesamiento local.
 - **452:** No se produjo la acción por que el disco no tiene espacio de almacenamiento suficiente.
 - **500:** Error en la sintaxis, no se pudo reconocer el comando.
 - **501:** Error en la sintaxis de los parámetros del comando.
 - **552:** Abandono de la acción porque se superó la reserva de espacio.
- **NOOP** (no operación):

Este comando provoca que el servidor responda con un OK.

No afecta a ningún comando enviado anteriormente o posteriormente. Se suele usar para asegurarse de que la conexión permanece activa.

Sintaxis: NOOP#13#10

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
- **421:** El servicio no está disponible.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.



- **QUIT:**

Este comando le indica al servidor que el cliente no tiene más operaciones que realizar y que se debería cerrar la conexión.

El servidor responde OK y seguidamente, cierra la conexión con el cliente.

Sintaxis: `QUIT#13#10`

Los posibles códigos de respuesta a este comando son:

- **221:** Se está cerrando la conexión.
 - **500:** Error en la sintaxis, no se pudo reconocer el comando.
- **RCPT (destinatario):**

Este comando indica al servidor quien es el destinatario del mensaje que se está enviando.

Si el mensaje debe de ir a varios destinatarios, se pueden expresar separados por comas.

La sintaxis del comando es:

```
RCPT TO: nombre_destinatario@host_destinatario  
[,nombre_destinatario@host_destinatario, ...]#13#10
```

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
- **251:** El usuario no es local, entonces se remite el mensaje a nombre-servidor.
- **421:** El servicio no está disponible.
- **450:** No se realizó la acción porque el buzón no está disponible.
- **451:** Se abandonó la acción por un error de procesamiento local.
- **452:** No se produjo la acción por que el disco no tiene espacio de almacenamiento suficiente.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.



- **501:** Error en la sintaxis de los parámetros del comando.
 - **503:** Secuencia de comandos incorrecta.
 - **550:** La acción no se realizó por que no se ha encontrado el buzón.
 - **551:** El usuario no es local, el cliente debería conectarse a nombre-servidor.
 - **552:** Abandono de la acción porque se superó la reserva de espacio.
 - **553:** No se realizó la operación porque la sintaxis del nombre del buzón es incorrecta.
- **RSET (reinicio):**

Este comando le indica al servidor que abandone la transacción de correo actual, que descarte los datos introducidos del remitente, destinatario o el mensaje.

RSET provoca que se vacíen y reinicien todos los buffers y tablas de estado.

Sintaxis: `RSET#13#10`

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
 - **421:** El servicio no está disponible.
 - **500:** Error en la sintaxis, no se pudo reconocer el comando.
 - **501:** Error en la sintaxis de los parámetros del comando.
 - **504:** El parámetro del comando no está implementado.
- **SEND:**

Este comando se utiliza para enviar correo a la pantalla del terminal de la sesión actual del destinatario del mensaje. No se envía el mensaje al buzón del destinatario.

Este comando se utiliza fundamentalmente cuando se necesita enviar un mensaje crítico, por ejemplo, al administrador del sistema.

Si el destinatario no puede recibir el mensaje, bien porque no está en sesión, bien porque el terminal no acepta mensajes, etc. el servidor devolverá un código de respuesta al comando RCPT que debería seguir al comando SEND.



Sintaxis: `SEND FROM: nombre_remitente@host_remitente#13#10`

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
 - **421:** El servicio no está disponible.
 - **451:** Se abandonó la acción por un error de procesamiento local.
 - **452:** No se produjo la acción por que el disco no tiene espacio de almacenamiento suficiente.
 - **500:** Error en la sintaxis, no se pudo reconocer el comando.
 - **501:** Error en la sintaxis de los parámetros del comando.
 - **502:** El comando no está implementado.
 - **552:** Abandono de la acción porque se superó la reserva de espacio.
- **SOML (enviar o enviar por correo):**

Este comando funciona como el comando SEND, con la diferencia de que, si la pantalla del terminal de destinatario del mensaje no puede recibir, por el motivo que sea, el mensaje, el buzón de dicho usuario recibirá de forma automática el mensaje.

Sintaxis: `SOML FROM: nombre_remitente@host_remitente#13#10`

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
- **421:** El servicio no está disponible.
- **451:** Se abandonó la acción por un error de procesamiento local.
- **452:** No se produjo la acción por que el disco no tiene espacio de almacenamiento suficiente.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.
- **501:** Error en la sintaxis de los parámetros del comando.
- **502:** El comando no está implementado.
- **552:** Abandono de la acción porque se superó la reserva de espacio.



- **SAML (enviar y enviar por correo):**

Este comando funciona igual que el comando SOML, con la diferencia de que siempre se envía el mensaje al buzón independientemente de que llegue a la pantalla del terminal o no.

Sintaxis: SAML FROM: nombre_remitente@host_remitente#13#10

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
- **421:** El servicio no está disponible.
- **451:** Se abandonó la acción por un error de procesamiento local.
- **452:** No se produjo la acción por que el disco no tiene espacio de almacenamiento suficiente.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.
- **501:** Error en la sintaxis de los parámetros del comando.
- **502:** El comando no está implementado.
- **552:** Abandono de la acción porque se superó la reserva de espacio.

- **TURN:**

Este comando invierte los papeles del servidor y del cliente.

El cliente toma el papel de destinatario y el servidor toma el papel de remitente.

Este comando se usa para recibir los mensajes de correo que desde el servidor se quiera enviar sin tener que esperar a que el servidor inicie una sesión SMTP con el cliente después de terminar la actual.

Sintaxis: TURN#13#10

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.
- **502:** El comando no está implementado.
- **503:** Secuencia de comandos incorrecta.



- **VRFY (verificar):**

Este comando le indica al servidor que verifique que el destinatario especificado, sea un usuario real y válido, por tanto, del sistema servidor.

Este comando se utiliza antes de iniciar un nuevo mensaje de correo.

Sintaxis: `VRFY nombre_usuario#13#10`

Los posibles códigos de respuesta a este comando son:

- **250:** La acción solicitada se ha completado.
- **251:** El usuario no es local, entonces se remite el mensaje a nombre-servidor.
- **421:** El servicio no está disponible.
- **500:** Error en la sintaxis, no se pudo reconocer el comando.
- **501:** Error en la sintaxis de los parámetros del comando.
- **502:** El comando no está implementado.
- **504:** El parámetro del comando no está implementado.
- **550:** La acción no se realizó porque no se ha encontrado el buzón.
- **551:** El usuario no es local, el cliente debería conectarse a nombre-servidor.
- **553:** No se realizó la operación porque la sintaxis del nombre del buzón es incorrecta.

RFC 1651

Estos son los comandos de la especificación del SMTP. Pero este es un protocolo que lleva funcionando ya muchos años y se han realizado algunas extensiones.

Ninguna de las extensiones se ha popularizado, pero sí que se ha estandarizado una de estas extensiones en la RFC 1651, cuyo objetivo es permitir que el cliente de correo electrónico, pueda conocer cuáles son las extensiones que soporta un servidor SMTP concreto.

El comando que permite esta posibilidad es **EHLO**, que se utiliza en lugar de **HELO** y con la misma sintaxis.



Si el servidor no admite ninguna extensión, responde con un código de respuesta 500, es decir, no reconoce el comando. En el caso de que sí reconozca el comando, responde con un código 250 y si admite alguna extensión, envía varias líneas que ofrecen una lista de las extensiones que soporta dicho servidor (cada línea comienza con el código de respuesta 250).

La seguridad en este protocolo:

Ni el conjunto de comandos SMTP ni el modelo de comunicación de este protocolo, disponen de un comando de entrada a un sistema de correo.

Por ello, cualquier cliente de correo puede conectarse a cualquier servidor y enviar mensajes de correo, lo que hace que el sistema de correo de Internet sea vulnerable.



+ Info

Cualquier usuario de Internet puede enviar correo con una dirección de remitente falsa o real de otra persona.

Aunque para hacerlo, hay que tener un conocimiento de estos protocolos y además hay que tener en cuenta que siempre hay ficheros logs, es decir, de registro de operaciones, mediante las cuales se pueden hacer rastreos.

QMTTP: protocolo rápido de transferencia de correo

Es un protocolo más avanzado, pero menos conocido.

El Protocolo rápido de transferencia de correo (QMTTP **Quick Mail Transfer Protocol**) es un protocolo de transmisión de correo electrónico diseñado para tener un mejor rendimiento que el Protocolo simple de transferencia de correo (**SMTP**), el estándar de facto.

Fue diseñado e implementado por Daniel J. Bernstein. **QMTTP** se puede usar sobre TCP. Un servidor **QMTTP** sobre TCP escucha conexiones TCP en el puerto 209.

2.2. POP

POP (Post Office Protocol o Protocolo de la oficina de correo). POP3 es su versión más utilizada.

POP3 se usa en clientes locales de correo para obtener los mensajes de correo electrónico almacenados en un servidor remoto.



Utiliza los siguientes puertos:

- El puerto 110 para conexiones no encriptadas.
- El puerto 995 para las encriptadas.

Es un protocolo de la capa de aplicación en el Modelo OSI.

POP3 está diseñado para recibir correo. No sirve para enviarlo.

Permite a los usuarios descargar su correo electrónico mientras tienen conexión y revisarlo posteriormente sin conexión.

Cabe mencionar que la mayoría de los clientes de correo incluyen la opción de dejar una copia de los mensajes en el servidor.

Cuando un cliente utiliza POP3 realiza los siguientes pasos:

- Se conecta.
- Obtiene todos los mensajes.
- Los almacena en la computadora del usuario como mensajes nuevos.
- Los elimina del servidor.
- Se desconecta.

POP 3 es un protocolo de la capa de aplicación (del modelo OSI) que utiliza TCP como protocolo de capa de transporte. Como dijimos, este protocolo está pensado para recuperar mensajes de correo accediendo al servidor en forma dinámica desde una workstation.

2.2.1. Operación básica

Para establecer una conexión a un servidor POP, el cliente de correo abre una conexión TCP en el puerto 110 del servidor.

Cuando la conexión se ha establecido, el servidor POP envía al cliente POP una invitación y después las dos máquinas se envían entre sí otras órdenes y respuestas que se especifican en el protocolo.

Como parte de esta comunicación, al cliente POP se le pide que se autentifique (Estado de autenticación), donde el nombre de usuario y la contraseña del usuario se envían al servidor POP. Si la autenticación es correcta, el cliente POP pasa al Estado de transacción, en este estado se pueden utilizar órdenes LIST, RETR y DELE para mostrar, descargar y eliminar mensajes del servidor, respectivamente.



Los mensajes definidos para su eliminación no se quitan realmente del servidor hasta que el cliente POP envía la orden QUIT para terminar la sesión. En ese momento, el servidor POP pasa al Estado de actualización, fase en la que se eliminan los mensajes marcados y se limpian todos los recursos restantes de la sesión.

Puedes conectarte manualmente al servidor POP3 haciendo Telnet al puerto 110 (ten en cuenta que este protocolo usa por defecto el puerto 23). Es muy útil cuando te envían un mensaje con un fichero muy largo que no quieres recibir.

Los comandos son:

- **USER** Identificación de usuario (Solo se realiza una vez).
- **PASS** Envías la clave del servidor.
- **STAT** Da el número de mensajes no borrados en el buzón y su longitud total.
- **LIST** Muestra todos los mensajes no borrados con su longitud.
- **RETR** Solicita el envío del mensaje especificando el número (no se borra del buzón).
- **TOP** Muestra la cabecera y el número de líneas requerido del mensaje especificando el número.
- **DELE** Borra el mensaje especificando el número.
- **RSET** Recupera los mensajes borrados (en la conexión actual).
- **QUIT** Salir.

2.2.2. Estado de "Autorización"

Establecida la conexión TCP solicitada por el cliente, el servidor de POP 3 envía el "saludo" que puede ser:

```
s: +OK POP3 server ready
```

Ahora el cliente debe identificarse ante el servidor, puede usar 2 tipos de mecanismos, que son mecanismos de autenticación, y el servidor está obligado a soportar como mínimo alguno de ellos. Son los siguientes:

- A través de los comandos de **USER** y **PASS**.
- O por el comando **APOP**.



Ante los comandos mencionados, el servidor debe enviar como respuesta:

- Un +OK: el usuario y password correctos.
- O un -ERR: el usuario y/o password son incorrectos.

Otro comando utilizado en esta etapa es el **QUIT**, que el usuario puede enviar cuando el servidor no termina por sí solo la conexión, ante el envío de un indicador de estado negativo de la casilla por el cual no se le puede dar acceso al cliente.

Después de que el servidor ha abierto y desactivado el estado negativo de la casilla del lado cliente y a la vez ha cerrado la entrada para evitar el ingreso de nuevos mails durante este período, asigna un número de mensaje a cada mensaje y declara el tamaño de cada uno en octetos. Aquí ya estamos en el estado de transacción.

2.2.3. Estado de "Transacción"

Si el resultado de la autenticación es correcto, se pasa al Estado de transacción, donde se utilizan órdenes para mostrar, descargar y eliminar mensajes del servidor.

Los mensajes definidos para su eliminación no se quitan realmente del servidor hasta que el cliente POP envía la orden QUIT para terminar la sesión.

Tras recibir cada orden, el servidor POP3 envía una respuesta. (el cliente puede enviar la orden QUIT y la sesión POP3 entrara en fase de Actualización).

Comandos que el cliente puede utilizar en este estado:

- **STAT**: el servidor puede responder positivamente con una línea conteniendo la cantidad de mensajes en la casilla y la cantidad de octetos total de todos los mensajes.

Ejemplo:

c: STAT.

s: + OK 2 320.

- **LIST (msg)**: Si el server responde positivamente, envía una línea conteniendo información sobre el mensaje determinado en el argumento del comando.

Ejemplo:

c: LIST

s: +OK 2 messages (320 octets)

s: 1 120

s: 2 200



.....

c: LIST 2

s: +OK 2 200

....

c: LIST 3

s: -ERR no such message, only 2 messages in maildrop

- **RETR** msg: El servidor ante una respuesta positiva envía el mensaje indicado en el argumento del comando.

Ejemplo:

c: RETR 1

s: +OK 120 octets.

s: "the pop 3 server sends the entire message here".

s: ...

- **DELE** msg: con esto el server marca el mensaje a borrar, pero realmente no lo hará hasta que la sesión de POP3 entre en el estado de "ACTUALIZACION". Cualquier referencia posterior al mensaje será errónea.

Ejemplo:

c:DELE 1.

s:+OK message 1 deleted.

Comandos opcionales en el estado de "Transacción"

Existe una serie de comandos opcionales que permiten manejar con mayor soltura los mensajes:

- Comando **TOP**.

sintaxis: TOP msg n

Tras el +OK inicial, el servidor de POP3 envía el encabezado del mensaje, una línea blanca separando el encabezado del cuerpo y luego el número de líneas del cuerpo del mensaje indicadas en el argumento del comando.

**Ejemplo:**

c: TOP 1 10

s: + OK

s: "the POP3 servers sends the headers of the message, a blank line, and the first 10 lines of the body of the message"

- Comando **UIDL**.

Sintaxis: UIDL (msg)

Da como resultado una línea que contiene el id único del mensaje indicado en el argumento del comando.

Este id único es un string determinado por el mismo server que consta de 1 a 70 caracteres en el rango 0x21 a 0x7E, que identifica en forma única un mensaje dentro del buzón y que persiste durante toda la sesión.

2.2.4. Estado de "Actualización"

Cuando el cliente envía el comando QUIT del estado de "Transaccion", la sesión POP3 entra en el estado de "Actualización".

Si la sesión termina por cualquier otra razón distinta del comando QUIT, la sesión no entra en el estado "Actualización" y no debe remover ningún mensaje de la casilla.

En el estado de "Actualización", se eliminan los mensajes marcados como "borrados", se limpian todos los recursos restantes de la sesión y cierra la conexión TCP.

2.2.5. Comandos que brindan seguridad

Existen comandos que brindan mayor seguridad sobre todo en la etapa de "AUTORIZACION" que es la más crítica dado que en ella es necesario dar el nombre de usuario y password al server.

- Comando **APOP**.

Normalmente, cada sesión POP3 comienza con el intercambio de USER/PASS, lo que hace que tanto el usuario y password se envíen en forma clara por la red. Si esto no se realiza frecuentemente el riesgo es mínimo, pero, si el uso es frecuente, aumenta el riesgo de la posible captura del password.



Para evitar este riesgo, se utiliza el comando APOP dando seguridad al emisor tanto con la autenticación como con la protección en la respuesta.

Sintaxis: APOP name digest

Un server que maneje APOP deberá incluir un string en el mensaje de bienvenida, que corresponde a una identificación de mensaje y debe ser diferente cada vez que se inicie una sesión POP3.

Este string sigue un algoritmo MD5 y la sintaxis es:

`"process-ID.clock@hostame"`

- Donde "proces -id" es un valor decimal del proceso PID (en UNIX).
- Clock es el valor decimal del sistema de reloj.
- Y hostname es el nombre de dominio del servidor sobre el cual corre POP3.

Los parámetros de este comando tienen una semántica idéntica a la del parámetro "name" del comando USER. El parámetro "digest" se calcula aplicando un algoritmo MD5 al string que representa al timestamp, seguido de una clave secreta que conocen el servidor y el cliente. Se debe tener gran cuidado para prevenir el conocimiento de este secreto, ya que conociéndolo permitirá a cualquier entidad enmascararse como el usuario que pretende ser. El "digest" tiene en sí 16 octetos que se envían en formato hexadecimal con caracteres ASCII minúsculos.

Cuando el servidor POP3 recibe el comando APOP, verifica el digest provisto. Si es correcto envía una respuesta positiva, entrando así en el estado de TRANSACCION. Si es negativa permanece en el estado de AUTORIZACIÓN.

La clave secreta debe tener una longitud tal que se llega a una solución de compromiso entre la dificultad de descifrarla por algún atacante y la dificultad en escribirla por el usuario.

Ejemplo:

S: +OK POP3 server ready <1896.697170952"deb.mtview.ca.us>

C: APOP mrose c4c9334bac560ecc979e58001b3e22fb

S: +OK maildrop has 1 message (369 octets)

En el ejemplo la clave secreta es el string "tan_staaf", de modo que aplicando el algoritmo MD5 al string.



<1896.697170952"deb.mtview.ca.us>tanstaaff

Produce el valor:

c4c9334bac560ecc979e58001b3e22fb

- Comando **AUTH**.

Solamente se utiliza durante el estado de "Autorización".

Sirve para indicar un mecanismo de autenticación al server, establecer un intercambio de protocolo de autenticación y opcionalmente negociar un mecanismo de protección para protocolos subsecuentes. Estos mecanismos también son utilizados por IMAP4.

Sintaxis: AUTH mechanism

Indica al server un mecanismo de autenticación. Si el server lo soporta se produce un intercambio entre server y usuario a través de un protocolo de autenticación para "autenticar e identificar al usuario". El intercambio depende exclusivamente del mecanismo de autenticación utilizado.

Cuando el servidor responde que está listo lo hace con una línea con "+", un espacio en blanco y un string codificado en BASE 64. El cliente también contesta con un string codificado en BASE 64.

Si el mecanismo solicitado fue aceptado, este mecanismo se aplica a todos los datos siguientes.

Ejemplo:

S: +OK POP3 server ready

C: AUTH KERBEROS_V4

S: + AmFYig==

S: + or//EoAADZI=

C: DiAF5A4gA+oOIALuBkAAmw==

S: +OK Kerberos V4 authentication successful

2.2.6. Inseguridades del protocolo POP

Es posible que para algunos usuarios (para algunas casillas en realidad) se permita la secuencia de comandos USER/PASS o el comando APOP, pero no ambos.

A pesar de tener clave secreta utilizando el comando APOP, cuanto más larga más segura pero también más difícil para escribirla.



Aquellos servers que dan respuesta –ERR al comando USER, dan pistas a posibles atacantes sobre los nombres válidos.

Utilizando el comando PASS se envían passwords en forma clara a través de la red.

Este protocolo no puede leer ningún header donde se indica el originador del mensaje o el lugar desde donde se realizó.

2.3. IMAP

IMAP (Internet Message Access Protocol).

IMAP es un protocolo de red de acceso a mensajes electrónicos almacenados en un servidor.

Utiliza los siguientes puertos:

- El puerto 143 para conexiones no encriptadas.
- El puerto 993 para las encriptadas con TLS.

Mediante IMAP se puede tener acceso al correo electrónico desde cualquier equipo que tenga una conexión a Internet.

Ventajas de IMAP sobre POP:

- Es posible especificar en IMAP carpetas del lado servidor.
- Permite visualizar los mensajes de manera remota sin descargarlos.

2.4. MIME

MIME (Multipurpose Internet Mail Extensions, Extensiones Multipropósito de Correo de Internet).

MIME es un estándar propuesto en 1991 por Bell Communications para expandir las capacidades limitadas del correo electrónico y en particular para permitir la inserción de documentos (como imágenes, sonido y texto) en un mensaje.

En sentido general las extensiones de MIME van encaminadas a soportar:

- Texto en conjuntos de caracteres distintos de US-ASCII.
- Adjuntos que no son de tipo texto.



- Cuerpos de mensajes con múltiples partes (multi-part).
- Información de encabezados con conjuntos de caracteres distintos de ASCII.

Los mensajes de correo electrónico en Internet están tan cercanamente asociados con el SMTP y MIME que usualmente se les llama mensaje SMTP/MIME.

2.5. S/MIME

S/MIME es un acrónimo que hace referencia a "extensiones seguras /multipropósito de correo en Internet ("Secure/Multipurpose Internet Mail Extensions", por sus siglas originales en inglés).

S/MIME es una tecnología que le permite cifrar correos electrónicos.

S/MIME está basado en los principios de la criptografía asimétrica que utiliza un par de claves matemáticamente relacionadas –una pública y otra privada– para funcionar.

Su finalidad es proteger los correos electrónicos frente a accesos no deseados.

Además, esta tecnología le permite firmar digitalmente los correos electrónicos para autenticarse como el remitente legítimo del mensaje, lo cual la convierte en una eficaz arma contra los numerosos ataques de phishing, que se producen cada día en Internet.

2.6. Otros protocolos

2.6.1. RFC 2142

Otro protocolo que debemos conocer es el **RFC 2142** es un protocolo utilizado para definir nombres de correo obligatorios en una organización privada.

El protocolo **RFC 2142** especifica un protocolo de seguimiento de estándares de Internet para la Comunidad de Internet, y solicita discusión y sugerencias para mejoras.



Resumiendo

Esta especificación enumera y describe las direcciones de correo de Internet (nombre del buzón @ referencia del host) que se utilizarán al contactar al personal de una organización.



Los nombres de los buzones se proporcionan tanto para operaciones como para funciones comerciales.

No se prohíben nombres y alias de buzones adicionales, pero se recomienda a las organizaciones que admiten intercambios de correo electrónico con Internet que apoyen "al menos" cada nombre de buzón para el que existe la función asociada dentro de la organización.

El **RFC 2142** (Request for Comments 2142), titulado "**Mailbox Names for Common Services, Roles and Functions**", establece convenciones estándar para la nomenclatura de buzones de correo electrónico en organizaciones. Fue publicado en **mayo de 1997** y es ampliamente utilizado en la administración de sistemas y redes para garantizar coherencia y facilidad de contacto en roles específicos.

El documento define un conjunto de nombres estándar de buzones para facilitar la comunicación con roles y servicios comunes dentro de un dominio de Internet. La idea es que las personas que necesiten contactar con un servicio o rol específico puedan hacerlo fácilmente al usar direcciones de correo predecibles.

2.6.2. Justificación y Alcance

Varios documentos de Solicitud de Comentarios o RFCs recomiendan nombres de buzones específicos para el uso de servicios determinados; por ejemplo, (RFC5321, 4.5.1) requiere la presencia de un nombre de buzón <**POSTMASTER**@domain> en todos los hosts que tienen un servidor SMTP.

Otros protocolos tienen estándares de facto para nombres de buzones conocidos, como:

- <**USENET**@domain> para NNTP, consulte RFC 977.
- Y <**WEBMASTER**@domain> para HTTP, RFC 2142.

También existen estándares predeterminados para nombres de buzones conocidos que no tienen nada que ver con un protocolo en particular, por ejemplo:

- <**ABUSE**@domain>, RFC 2142.

El propósito de este protocolo es agregar y especificar el conjunto básico de nombres de buzones que las organizaciones deben admitir. La mayoría de las organizaciones no necesitan admitir el conjunto completo de nombres de buzones definidos aquí, ya que no todas las organizaciones implementarán todos los servicios asociados. Sin embargo, si se ofrece un servicio determinado, se deben admitir los nombres de buzón asociados, lo que da como resultado la entrega a un destinatario apropiado para el servicio o rol referenciado.



Si un host no está configurado para aceptar correo directamente, pero implementa un servicio para el cual esta especificación define un nombre de buzón, ese host debe tener un conjunto MX RR y los intercambiadores de correo especificados. Por este conjunto RR deben reconocer el nombre de dominio del host al que se hace referencia como "local" con el fin de aceptar el correo enlazado para el nombre del buzón definido.

Esto ocurre incluso si el nombre de dominio anunciado no es el mismo que el nombre de dominio del host.



Ejemplo

Si el nombre de host de un servidor NNTP es DATA.RAMONA.VIX.COM pero anuncia el nombre de dominio VIX.COM en sus encabezados "Ruta:", entonces el correo debe poder entregarse tanto a <USENET@VIX.COM> como a <USENET@DATA.RAMONA.VIX.COM>, aunque estas direcciones puedan ser entregadas a diferentes destinos finales.

El alcance de un nombre de buzón conocido es su nombre de dominio.

Los servidores que aceptan correo en nombre de un dominio deben aceptar y procesar correctamente los nombres de los buzones de ese dominio, incluso si el servidor en sí no admite el servicio asociado.



Ejemplo

Si un servidor NNTP anuncia el dominio de nivel superior de la organización en los encabezados "Path:", los intercambiadores de correo para ese dominio de nivel superior deben aceptar el correo a <USENET@domain> incluso si el intercambiador de correo aloja ellos mismos no sirven el protocolo NNTP.



2.6.3. Invariantes

Para nombres conocidos que no están relacionados con protocolos específicos, solo se requiere que el nombre de dominio de nivel superior de la organización sea válido.



Ejemplo

Si el nombre de dominio de un proveedor de servicios de Internet es COMPANY.COM, entonces la dirección <ABUSE@COMPANY.COM> debe ser válida y admitida, a pesar de que los clientes cuya actividad genera quejas usan hosts con nombres de dominio más específicos como SHELL1.COMPANY.COM.

Sin embargo, es válido y se recomienda admitir nombres de buzones para subdominios, según corresponda.

Los nombres de los buzones deben reconocerse independientemente del tipo de caracteres, han de ser case insensitive. Por ejemplo, POSTMASTER, postmaster, Postmaster, PostMaster e incluso PoStMaStEr deben ser tratados de la misma manera, con entrega al mismo buzón.

Las implementaciones de estos nombres bien conocidos, deben tener en cuenta las expectativas de los remitentes que los usarán.

Enviar un acuse de recibo de correo automático suele ser útil (aunque sugerimos precaución contra la posibilidad de "enfrentarse a robots de correo" y los bucles de correo resultantes).

2.6.4. Nombres de buzones

2.6.4.1. Nombres de Buzones Relacionados con la Empresa

Estos nombres están relacionados con las actividades de línea de negocio de una organización. El nombre INFO a menudo está vinculado a un autoresponder, con una gama de archivos estándar disponibles.

Mailbox	AREA	USO
INFO	Marketing	Información empaquetada sobre la organización, productos y / o servicios, según corresponda



Mailbox	AREA	USO
MARKETING	Marketing	Marketing de productos y Comunicaciones de marketing
SOPORTE DE VENTAS	SERVICIO DE VENTA AL CLIENTE	Problemas de información de compra del producto con el producto o servicio

2.6.4.2. Nombres de Correos de Red

Las direcciones de operaciones están destinadas a proporcionar recursos a clientes, proveedores y otras personas que están experimentando dificultades con el servicio de Internet de la organización.

Mailbox	AREA	USO
ABUSE	Relaciones con los clientes	Comportamiento público inapropiado
NOC	Operaciones de RED	Operaciones de RED
SECURITY	Seguridad de RED	Boletines de seguridad o consultas

2.6.4.3. Nombres de Correos de Apoyo para Servicios de Internet Específicos

Para los principales servicios de protocolo de Internet, hay un buzón definido para recibir consultas e informes: si una organización proporciona servicios que utilizan los protocolos SMTP, FTP y HTTP, sería recomendable tener buzones de correo electrónico específicos para cada uno de estos servicios. (Aquí se incluyen los sinónimos, debido a su extensa base instalada).

Mailbox	SERVICIO	PROTOCOLOS
POSTMASTER	SMTP	RFC821, RFC822
HOTMASTER	DNS	RFC1033-RFC1035
USENET	NNTP	RFC977
NEWS	NNTP	Sinónimo de USENET
WEBMASTER	HTTP	RFC2068
WWW	HTTP	Sinónimo de WEBMASTER
UUCP	UUCP	RFC976
FTP	FTP	RFC959



2.6.4.4. Lista de Correo: Buzón de Administración

Las listas de correo tienen un nombre de buzón administrativo al que se pueden enviar solicitudes de agregar / quitar y otras metaconsultas.

Para una lista de correo cuyo nombre de buzón de envío es:

- <LIST@DOMINIO>.

DEBE haber el nombre del buzón administrativo:

- <LIST-REQUEST@DOMINIO>.

El software de administración de la Lista de distribución, como MajorDomo y Listserv, también tiene un solo nombre de buzón asociado con el software en ese sistema, generalmente el nombre del software, en lugar de una lista particular en ese sistema. El uso de dichos nombres de buzones requiere que los participantes conozcan el tipo de software de lista empleado en el sitio.

Esto puede ser problemático, por lo que:

Se requieren nombres de correo específicos de lista (-Request), independientes de la disponibilidad de nombres de software de la lista genérica.

2.6.5. Administración del Buzón de Nombres de Dominio

En DNS, el registro de inicio de autoridad (SOA RR) tiene un campo para especificar el nombre del buzón del administrador de la zona.

Este campo debe ser una palabra simple sin metacaracteres (como "%" o "!" o ":"), y se debe usar un alias de correo en los hosts del intercambiador de correo relevante para dirigir el correo de administración de zona al buzón apropiado.

Por simplicidad y regularidad, se recomienda encarecidamente que el conocido nombre de buzón HOSTMASTER se use siempre <HOSTMASTER@dominio>.

2.6.6. Correos de los Sistemas Autónomos

No existe una orden que obligue a los SA (Servicio Autónomo) a tener un buzón de correo electrónico por cada servicio que ofrece. Puede tener buzones diferentes para diferentes servicios, un único buzón para todos los servicios o incluso carecer de buzones de correo electrónico.

Un RIR (Regional Internet Registry o Registro Regional de Internet) se basará en los estándares RFC para determinar el contacto administrativo de un SA en base a un servicio ofrecido por este último, sin embargo en algunos casos, como decíamos, un SA puede carecer de dicho buzón con la consiguiente pérdida de los correos destinados al mismo. Un extremo que debe tratarse como un inconveniente más que como un error o una violación de los estándares.



2.6.7. Consideraciones de Seguridad

Los ataques de denegación de servicio ("Denial of service attacks" inundar un buzón con basura) serán más fáciles después de que este protocolo se convierta en un estándar, ya que más sistemas admitirán el mismo conjunto de nombres de buzones.

3. Administración del correo electrónico

3.1. Monitorización



Fuente: (<https://pixabay.com/es/gran-hermano-monitoreo-2783030/>)

La monitorización de servidores consiste en la vigilancia de los servicios activos que un servidor nos ofrece.

El servicio de correo electrónico es la herramienta imprescindible para la comunicación entre la empresa y sus clientes, proveedores, colaboradores y miembros de la empresa.

Por ello es muy importante garantizar su correcto funcionamiento.

Debemos disponer de mecanismos que garanticen que los correos que enviamos son recibidos por el destinatario, sin interferencias ni barreras.

Si contratamos un servicio de mensajería externo, este problema no es nuestro.

Sin embargo, si montamos nuestros propios servidores de correo, necesitaremos alguna herramienta de monitorización con sensores que nos den información en tiempo real sobre el funcionamiento de los servicios.

En caso de mal funcionamiento deberá alertarnos.



Existen programas de monitorización específicos para servidores de correo, pero se puede utilizar *software* de monitorización de redes, que incluyen también monitorización de servidores de correo.

Nosotros te proponemos varias alternativas:



El experto opina

Ten en cuenta, que con el tiempo quedarán obsoletas.

Nagios



Fuente: (<https://www.flickr.com/photos/xmodulo/11700273965>)

Es un software de monitorización de equipos y servicios de red, creado para ayudar a los administradores a tener siempre el control de qué está pasando en la red y conocer los problemas que ocurren en la infraestructura antes de que los usuarios los perciban.

Es un sistema complejo y completo en cuanto a sus características que además hace uso en algunos casos de diversos sistemas como por ejemplo sistemas gestores de bases de datos, servidores web, etc.

Está implementado en lenguaje PHP.

Puede utilizar el protocolo SNMP, que también sirve para monitorizar espacio en disco, carga de CPU y memoria libre.



O NRPE (Nagios Remote Plugin Executor), que permite ejecutar plugins en una máquina remota.

Nagios está licenciado bajo la GNU General Public License Version 2.

A continuación presentamos los comandos esenciales de Nagios:

Comandos de Control del Servicio:

- `systemctl start nagios`: inicia el servicio de Nagios.
- `systemctl stop nagios`: detiene el servicio de Nagios.
- `systemctl restart nagios`: reinicia el servicio de Nagios.
- `systemctl status nagios`: verifica el estado del servicio de Nagios.

Comandos de Configuración y Verificación:

`nagios -v /etc/nagios/nagios.cfg`: verifica la sintaxis y validez del archivo de configuración de Nagios.

`nagios -v`: muestra la versión de Nagios instalada en el sistema.

`cat /usr/local/nagios/var/nagios.log` → Muestra los registros de eventos y errores de Nagios.

Comandos para Manejo de Objetos y Hosts:

- `htpasswd -c /usr/local/nagios/etc/htpasswd.users <usuario>`: crea un nuevo usuario para acceder a la interfaz web de Nagios.
- `htpasswd /usr/local/nagios/etc/htpasswd.users <usuario>`: modifica la contraseña de un usuario existente.
- `service nagios reload`: recarga la configuración de Nagios sin reiniciar el servicio.

Comandos de Notificación y Estados:

- `tail -f /usr/local/nagios/var/nagios.log` → Monitorea en tiempo real los eventos del sistema de Nagios.
- `nagios -s` → Muestra estadísticas del sistema de monitoreo.

Comandos de Plugins y Chequeos Manuales:

- `/usr/local/nagios/libexec/check_ping -H <host> -w 100.0,20% -c 500.0,60%` → Realiza un chequeo manual de ping a un host.
- `/usr/local/nagios/libexec/check_http -H <host>` → Verifica el estado de un servicio web en un host específico.
- `/usr/local/nagios/libexec/check_disk -w 20% -c 10% -p /dev/sda1` → Comprueba el espacio en disco de una partición específica.



Check_MK



Fuente (<https://checkmk.com/company/brand-assets>)

CheckMK es una herramienta que se construye sobre la base de Nagios, y al igual que Nagios es un software de monitorización para sistemas Linux y Unix. Algunos lo consideran como una extensión de Nagios con mejoras en la usabilidad y funcionalidades añadidas. Por su versatilidad puede ser implementado sobre subredes de Modelo Clase A, Modelo Clase B, Modelo Clase C o incluso en redes de enrutamiento sin clase (CIDR).

Algunas de sus mejores con respecto a Nagios son una configuración con interfaz web, un motor de monitoreo eficiente y con un menor consumo de recursos del sistema, la identificación automática de nuevos hosts o servicios de red, notificaciones de estado en tiempo real que permiten una respuesta inmediata a eventos críticos.

Pandora FMS



Fuente:
(https://commons.wikimedia.org/wiki/File:Logo_Pandora_FMS_community_edition.png)

Pandora FMS es un *software* de monitorización para gestión de infraestructura TI.

Esto incluye:

- Equipamiento de red.
- Servidores Windows.
- Servidores Unix.
- Infraestructura virtualizada.
- Aplicaciones.



Pandora FMS tiene multitud de funcionalidades, lo cual lo convierte en un *software* de nueva generación que cubre todos los aspectos de monitorización necesarios.

Es un *software* de código abierto.

3.2. Seguridad



Fuente: (<https://pixabay.com/es/seguridad-cibern%C3%A9tica-protecci%C3%B3n-3400657/>)



+ Info

Puedes consultar algunas buenas prácticas publicadas por la Junta de Andalucía en su documento "Correo electrónico seguro".

También proponen otros enlaces interesantes.

http://www.formacion.andaluciaesdigital.es/c/document_library/get_file?uuid=3381a004-24d7-4d99-82bf-e47949cc80d7&groupId=20195

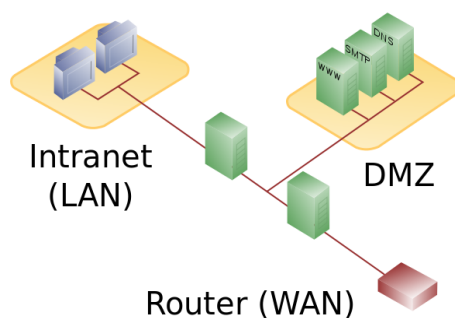
3.2.1. Entorno del servidor

El entorno del servidor debe ser seguro y no verse amenazado por las vulnerabilidades que pudieran existir en otras aplicaciones o servicios de la organización.

Para ello, es importante ubicar los servidores de correo, especialmente si estos tienen exposición a la red externa (Internet), en una zona de la red suficientemente aislada y controlada.



Habitualmente y según las recomendaciones del CCN-CERT, esta zona será la DMZ (DeMillitarized Zone, en castellano, zona desmilitarizada) de la red, convenientemente aislada del resto de las redes de la organización haciendo uso de cortafuegos que filtren los accesos desde Internet hasta nuestros servidores, así como desde estos al resto de redes de nuestra organización.



Fuente:
(https://es.wikipedia.org/wiki/Archivo:DMZ_net_work_diagram_2_firewall.svg)

Se debería considerar en la política de filtrado del firewall limitar todo el tráfico saliente SMTP únicamente a nuestro servidor de correo para evitar que otras aplicaciones puedan enviar correo directamente al exterior.

Así evitamos que algún otro servicio o equipo de usuario pueda verse utilizado como propagador de *malware*.

Como regla general, en nuestro cortafuegos deberíamos permitir únicamente el acceso a los servicios expresamente habilitados para el correo (SMTP, POP e IMAP) y denegar cualquier otro tipo de acceso a los servidores de correo.

Adicionalmente, estas reglas deberían también filtrar las redes desde las que se puede acceder a estos servicios, por ejemplo, limitando el acceso POP e IMAP únicamente a las redes de usuarios.

Igualmente, sería deseable que existiera un elemento de detección y/o prevención de intrusiones que permitiese monitorizar o incluso identificar y detener cualquier tipo de ataque a la plataforma de correo, así como registrar tráfico anómalo que pudiera producirse.

Es importante que los registros de estos sistemas sean analizados periódicamente por el equipo de seguridad para así verificar que el comportamiento de este elemento de seguridad sea el adecuado y que no se estén generando alertas que requieran algún tipo de revisión manual.

3.2.2. Configuración segura de los servidores

Los servicios de correo electrónico deben cumplir una serie de requisitos que deben estar definidos en nuestra política de seguridad.

Algunos requisitos podrían ser:

- Instalación únicamente de los servicios y aplicaciones necesarios para la prestación del servicio.
- El resto de los servicios se eliminarán o desactivarán.



- Instalar los parches y actualizaciones de las aplicaciones utilizadas. Esto implica informarse a diario de los parches y actualizaciones que van apareciendo.
- Política de gestión de usuarios control de accesos y permisos. Se debe garantizar la confidencialidad de la información almacenada.
- Monitorización continua y auditorías periódicas para mejorar los servicios.

3.2.3. Seguridad en servicios de correo



Fuente: (<https://pixabay.com/es/spam-casilla-de-correo-2636258/>)

Se deben establecer medidas de seguridad específicas para los servicios de correo electrónico que respondan a los problemas de seguridad que los afectan con más frecuencia.

Ataque típico Open Relay

Se denomina ataque por Open Relay al mecanismo de usar el MTA como puente para correos (usualmente spam) que de otra manera no podrían llegar a destino porque la IP de origen está bloqueada.

Estos servidores que permiten que se envíe correos a través de ellos, se los denomina Open Relay.

Para solucionar esto se crearon listas negras en tiempo real que bloquean dichos hosts en los cuales se detectó un MTA que hacía Open Relay.

Para que se saque una IP de estas listas negras, se deben pasar ciertas pruebas y esperar cierto tiempo.



+ Info

Hay que tener cuidado con esta práctica, ya que se culpa y castiga al propietario del servidor que lo permite (por acción u omisión) y no la persona que realiza el ataque.

Para solucionarlo, la configuración del servicio debería permitir únicamente el envío o clientes de correo autenticados, con lo que sólo se permitiría la utilización de la plataforma de envío a los usuarios legítimos del sistema.

3.2.4. Auditorías del sistema



Fuente:
(<https://pxhere.com/es/foto/1446123>)

Una vez establecidas las medidas de seguridad necesarias en nuestros sistemas, se debería realizar una auditoría de seguridad de la plataforma completa para verificar que se cumple nuestra política de seguridad.

Esta auditoría del sistema debería repetirse periódicamente.



3.2.5. Seguridad en clientes de correo

Los clientes de correo electrónico son un punto crítico por tres motivos:

- Están menos controlados.
- Es el principal punto de entrada de amenazas.
- Los usuarios no conocen bien los clientes de correo ni están concienciados (por norma general) con la política de seguridad.

Algunas de las medidas de seguridad a contemplar son:

- Los sistemas operativos y las aplicaciones deberían actualizarse de forma automática.
- Los usuarios no deben tener permisos de administrador.
- Deben tener un antivirus instalado.
- Los equipos de usuario deberían tener activado el servicio de firewall.

3.2.5.1. Clientes de correo

Los clientes de correo deben configurarse de manera segura y evitar que los usuarios modifiquen esta configuración.

Algunas medidas de seguridad en la configuración de los clientes de correo son:

- Utilizar de protocolos seguros (SSL/TLS) tanto para el envío como para la recepción de correos.
- Desactivar de la carga de contenidos externos y la reproducción automática de contenidos (JavaScript, ActiveX).
- No almacenar las contraseñas en la configuración del cliente de correo.

3.2.5.2. Correo WEB

Si se accede a correo web con navegadores, se deberían contemplar los siguientes aspectos:

- No permitir el autoguardado de contraseñas.
- Establecer una contraseña maestra para acceder a certificados o cualquier tipo de información privada.
- Desactivar la reproducción de contenido remoto dentro de los correos electrónicos.
- Limitar o deshabilitar la instalación de addons.
- Cerrar la sesión de manera automática y eliminar historial y cookies una vez se cierre la ventana del navegador.



3.2.6. Dispositivos móviles

Estos dispositivos deben ser tratados de igual manera que los equipos de escritorio y mantener las mismas medidas de seguridad.

Además, tenemos que considerar algunos riesgos adicionales:

- Estos dispositivos están más expuestos a riesgos como el extravío o el robo de los mismos, por lo que debemos asegurarnos de establecer algún mecanismo de control de acceso.
- Cifrar la información intercambiada con los servidores de correo (por si utiliza redes públicas).

Un ejemplo de programas especialmente diseñados para el correo en dispositivos móviles es el Microsoft Exchange.

Este servicio trabaja con el protocolo **ActiveSync**, **Exchange ActiveSync** es un protocolo de sincronización de Exchange que está optimizado para trabajar con redes de alta latencia y bajo ancho de banda. Este protocolo basado en HTTP y XML permite a los teléfonos móviles acceder a la información de la organización, Permite ejecutar una **"Remote Wipe"** (limpieza remota) para eliminar contenido del servidor Exchange del dispositivo conectado.

Exchange ActiveSync permite a los usuarios de teléfonos móviles acceder a su correo electrónico, calendario, contactos y tareas y les permite seguir teniendo acceso a esta información cuando trabajan sin conexión.

Los servicios de cifrado estándar agregan seguridad a las comunicaciones móviles con el servidor. Puede configurar Exchange ActiveSync para utilizar el cifrado de Capa de sockets seguros (SSL) a fin de establecer la comunicación entre el servidor Exchange y el dispositivo móvil.

3.2.7. Gestión de contraseñas



Fuente: (<https://pixabay.com/es/internet-de-seguridad-contrase%C3%B1a-1952019/>)

La gestión de usuarios y contraseñas es la primera línea de defensa frente al acceso no autorizado.

Debemos determinar una serie de condiciones de seguridad mínimas en cuanto a las características de las contraseñas, así como periodos de renovación de estas.



Todas estas condiciones y medidas de seguridad deberían estar definidas en la política de seguridad de la organización.

Las contraseñas deben ser robustas y secretas. Para ello:

- **Deben tener una longitud adecuada.** (A mayor número de caracteres, mayor dificultad de obtenerla con algoritmos de fuerza bruta).
 - Longitud mínima de ocho caracteres.
- **No deben utilizarse palabras conocidas:**
 - Palabras del diccionario.
 - Nombres propios.
 - Lugares.

Las contraseñas con estas palabras son más fáciles de romper con ataques de diccionario.

- **Las contraseñas deben incluir variedad de caracteres:**
 - Mayúsculas.
 - Minúsculas.
 - Números.
 - Signos de puntuación.
- **No se deben reutilizar claves.** Cada servicio que usemos debe tener una clave diferente.

Se deberían establecer políticas de seguridad que obliguen a los usuarios a cambiar la contraseña periódicamente, evitando la reutilización de estas.



Recomendación

Visita la siguiente web para que conozcas diferentes herramientas de administración de contraseñas.

[Gestores de contraseñas](#)



4. Administración de contenedores y microservicios

Los microservicios, que también se les conoce como la arquitectura de microservicios (MSA), es una forma diferente de desarrollar un software, que consiste en la creación de una aplicación con la unión de pequeños servicios independientes, que corren bajo sus propios procesos, y que debido a esa independencia deben comunicarse entre sí.

Normalmente se utiliza el protocolo HTTP vía API, ya que es más sencilla la configuración del firewall para permitir el funcionamiento de la aplicación.

La arquitectura de microservicios es una forma diferente de desarrollar un software, implementando los microservicios, ofreciendo un nuevo enfoque de desarrollo de software basado en la potenciación de la autonomía de cada servicio o funcionalidad.

La arquitectura de microservicios se basa en el concepto "haz una sola cosa y hazla bien".

En lugar de realizar un proyecto global como un todo, se distribuyen las tareas en componentes pequeños, estos son los denominados microservicios o microservices, concepto que se aplica tanto en la programación de una aplicación como en la planificación del proyecto, con el objetivo de agilizar así su gestión.

Esta división de la aplicación en componentes, más pequeños, hace que puedan ser mantenidos, ejecutados y distribuidos de forma totalmente independiente. Los microservicios permiten cambios rápidos y controlados en el software.



+ Info

El uso de los microservicios se basa en la filosofía Unix de Ken Thompson: "Do one thing and do it well", traducido como "haz una cosa y hazla bien".

Para la existencia de la arquitectura de microservicios son necesarios los contenedores y orquestadores.

En la arquitectura de microservicios, se puede aplicar la metodología de entrega continua (continuous delivery), que es una metodología por la cual algunos productos de software se mantienen en constante desarrollo. Los fabricantes no necesitan planificar ni gestionar grandes ciclos de actualizaciones, ya que un cambio en un microservicio, tras realizarse la fase de prueba, se publica directamente sin tener que depender del resto de procesos.



4.1. Que son los microservicios

En una arquitectura de microservicios cada función se llama servicio y se construye e implementa de forma independiente, por tanto, cada servicio puede funcionar (y fallar) sin afectar a los demás.



El uso de microservicios es un enfoque arquitectónico para la creación de aplicaciones, que deben ser independientes y acoplarse. Así el fallo en uno de ellos no afecta al resto al igual que los equipos de desarrollo ante nuevas necesidades del mercado, pueden construir rápidamente nuevos componentes de aplicaciones para satisfacerlas.

Movimiento tecnológico de devops

Gracias al sistema de individualidad de cada función, se puede adoptar el movimiento tecnológico de DevOps, consiguiendo los objetivos que pretende este movimiento:

- Automatización y el monitoreo en todos los pasos de la construcción del software.
Tanto las pruebas, el despliegue, la implementación y la administración de la infraestructura, se realiza de forma independiente para cada microservicio.
- Acortar el ciclo de vida del desarrollo de software.
- Mayor frecuencia de implementación.

Lanzamientos más efectivos, con una entrega continua de alta calidad (constant iteration and delivery o CI/CD).



+ Info

DevOps, acrónimo inglés de Development (desarrollo) y operations (operaciones).

Es un conjunto de prácticas que agrupan el desarrollo de software (Dev) y las operaciones de TI (Ops).

4.2. Ventajas de la arquitectura de microservicios

Al ser componentes modulares independientes, son más fáciles de probar, mantener y comprender. Lo que da como resultado un incremento de la agilidad, mejora de los flujos de trabajo y menor tiempo para mejorar la producción.

Algunas de las ventajas que ofrecen los microservicios:

4.2.1. Especialización del equipo

Puesto que cada microservicio realiza una tarea determinada buscando su perfección, facilita la especialización del equipo encargado de su desarrollo, lo que hace que pueda lograrse más fácilmente esa perfección.

4.2.2. Altamente escalable

Cada microservicio puede ser escalado independientemente de la aplicación completa, lo que posibilita una gran capacidad de expansión.

Se pueden lanzar nuevas instancias y/o eliminar las que ya no son necesarias. En función de la variación de la demanda de los servicios, estos pueden fortalecerse, añadir nuevos servicios o bien eliminarse fácilmente, adaptándose con facilidad a las cambiantes necesidades de rendimiento.

En un sistema monolito, se debe escalar el sistema completo, pero en la arquitectura de microservicios, los desarrolladores pueden fortalecer el servicio que lo requiere con un alto grado de exigencia y detalle, y si es necesario integrar un nuevo servicio, no se requiere tanto trabajo, debido a su independencia.

Si crece la demanda de ciertos servicios, es posible realizar implementaciones en distintos servidores e infraestructuras, satisfaciendo así dicha demanda.

De esta forma el producto final es mucho más ligero y necesita menos recursos.



4.2.3. Independencia y versatilidad, aplicaciones más abiertas

Los microservicios permiten el uso de diferentes tecnologías y lenguajes, se puede elegir el lenguaje de programación o la base de datos que se adapte mejor a la funcionalidad requerida (servicio) en lugar de tener que tomar una más estandarizada.

Con el uso de API políglotas (políglota proviene del griego, y significa "muchas lenguas"), los desarrolladores pueden elegir los mejores lenguajes y tecnologías para cada función.

Puesto que los equipos de trabajo del desarrollo de cada microservicio son independientes y no tienen necesidad de coordinarse entre sí continuamente, el equipo se centra en el perfecto desarrollo del microservicio.

Como cada servicio tiene su propio entorno de ejecución, es posible incluso utilizar lenguajes de programación diferentes para diversos microservicios o implementar bases de datos o sistemas para gestionarlas de desarrollo propio.

4.2.4. Consistencia: aislamiento de fallos y capacidad de recuperación

Como resultado de esa independencia de los microservicios el sistema se hace más robusto. Si se produce un error en un servicio, la aplicación completa no tiene por qué dejar de funcionar. La caída de una instancia puede superarse desplegando rápidamente nuevas instancias.

Al ser un proceso de tamaño menor, el error se puede encontrar mucho más fácilmente que en una aplicación de arquitectura monolítica.

Al ser independientes, el fallo de un microservicio no compromete la integridad y el buen funcionamiento de los demás, no afecta a toda la aplicación, lo que si ocurre en las aplicaciones monolíticas.

4.2.5. Rapidez de respuesta implementación y actualización

Puesto que se trata de partes más pequeñas, los desarrolladores pueden comprender, actualizar y mejorar más fácilmente cada microservicio, obteniéndose ciclos de desarrollo más rápidos, (especialmente si se utilizan metodologías de desarrollo ágiles).

Por tanto, se reducen los ciclos de desarrollo, y se agilizan los procesos de implementación y actualización de las aplicaciones. Se produce la entrega continua (continuous delivery).



5. Arquitectura de microservicios frente a la arquitectura monolítica

La gran ventaja que ofrece una arquitectura de microservicios es su agilidad. Y es que, antes de su aparición, todo se basaba en arquitecturas monolíticas y las distintas partes que conformaban un programa de software estaban acopladas y esto genera el problema de que, por ejemplo, todos esos programas no son escalables y es muy difícil añadir funcionalidades nuevas.

Una arquitectura de microservicios permite que los servicios acoplados libremente se puedan desarrollar, implementar y mantener de forma independiente. Cada uno de estos servicios es responsable de tareas discretas y puede comunicarse con otros servicios a través de APIs simples para resolver un problema comercial complejo más grande.

En una arquitectura de microservicios, como su propio nombre indica, los servicios son pequeños. La gran ventaja de ello es que gracias a ese tamaño pueden ser construidos por uno o más equipos pequeños desde el principio y puede ser separados por límites de servicio que facilitan la ampliación del esfuerzo de desarrollo si es necesario.

Las aplicaciones tradicionales tienen un enfoque monolítico, se agrupan todos los servicios en una aplicación, se escalan y se ejecutan en bloque, todo se integra en una única pieza, cada uno de los servicios recurre a una misma base de datos y se entrega por medio de una interfaz de usuario, todo en una única aplicación.

En cambio, los microservicios son independientes, parte de la idea de módulos, cada microservicio es responsable de una única tarea, se ejecutan de forma aislada (horizontalmente) y se comunican entre sí mediante APIs o recursos HTTP. Esta autonomía, permite, por ejemplo, escalar un área funcional concreta para poder ofrecer rápida respuesta a un repentino incremento de la demanda de un servicio o evitar que un fallo se propague de una instancia a otra.

Con los microservicios, se trata de concentrarse en una sola tarea y llevarla a la perfección, lo que se aplica en el trabajo de los programadores como y describe, como ya hemos indicado, la forma de funcionar de cada microservicio.

En la gestión de proyectos, se define y limita en qué se debe concentrar cada equipo y su independencia, sin supeditarse a una administración central. Por tanto, cada equipo es responsable de su propio producto final en cada fase de su ciclo de vida, dando lugar a una arquitectura de software modular, que ofrece numerosas ventajas.

Diferenciamos:

- En el desarrollo de una aplicación tradicional, como un monolito, todos los equipos dependen unos de otros.
- En una arquitectura de microservicios, se debe evitar esa interdependencia.

Se forman equipos pequeños, para que cada uno de ellos se ocupe de un solo servicio realizado en un microservicio.



Con este enfoque monolítico los equipos se organizan de forma diferente en función de la tecnología que utilizan:

- Mientras uno se dedica a las bases de datos.
- Otro se ocupa de programar los diversos servicios y otro se encarga de diseñar la interfaz de usuario.
- Otros grupos de trabajo son responsables de publicar actualizaciones, del mantenimiento y de la monitorización.



+ Info

La combinación de método de trabajo y producto se inspira en la teoría de Melvyn Conway, informático que en 1967 ("How Do Committees Invent?") que observó que las estructuras de los programas y sistemas reflejan siempre las estructuras del grupo que los desarrolla.

La arquitectura basada en microservicios es una evolución de la arquitectura orientada a servicios o SOA (Service-Oriented Architecture), pero en ella los servicios siguen estando integrados en un sistema mayor, y no tienen la autonomía que deben tener en una arquitectura de microservicios.

6. El cambio a microservicios

El objetivo de una empresa al querer convertir las aplicaciones monolíticas en microservicios, es la disminución del "time-to-market" (plazo de lanzamiento) de las aplicaciones, y de una mayor agilidad, con todas las ventajas que ya hemos visto que ofrecen los microservicios.



+ Info

El time to market o TTM es el tiempo que transcurre desde que se concibe un producto o servicio hasta que se lanza al mercado. Se trata entonces, de todos los procesos de trabajo que se generan para poder ofrecer ese producto o servicio al cliente final.

También es conocido como Speed to market.



El cambio a una arquitectura de microservicios implica como está desarrollada la aplicación, pero es algo más que la división de tareas, su individualidad y su acoplamiento, también y muy importante, la forma en que las personas trabajan, es necesario reestructurar los equipos de desarrollo y la coordinación entre ellos.

Este cambio organizativo en los equipos son un gran desafío, ya que cada equipo tendrá su propia cadencia de implementación y será responsable de un servicio.

Además, cada uno de estos equipos, puede elegir sus propias herramientas, puesto que los microservicios pueden comunicarse a través de Interfaces de Programación de Aplicaciones (APIs) independientes del lenguaje, ya que los microservicios pueden comunicarse entre sí, generalmente sin estado, por lo que las aplicaciones creadas de esta manera pueden ser más tolerantes a fallas y menos dependientes de un único ESB.

El estilo arquitectónico de microservicios se puede interpretar como una evolución de la Arquitectura Orientada a Servicios (SOA).



+ Info

ESB, o bus de servicio empresarial:

Es un patrón mediante el cual un componente de software centralizado realiza integraciones a sistemas de fondo (y traducciones de modelos de datos, conectividad profunda, direccionamiento y solicitudes) y hace que esas integraciones y traducciones estén disponibles como interfaces.

El desarrollo de aplicaciones con microservicios se ha popularizado y hecho más viable debido a los avances en las tecnologías de contenedorización, que permiten ejecutar varias partes de una aplicación de forma independiente.



+ Info

Normalmente, Java es el lenguaje de programación más elegido para desarrollar microservicios, pero pueden utilizarse otros como Golang y Python.



6.1. Desafíos (problemas) en la arquitectura de microservicios

Adoptar la arquitectura de microservicios, contenedores y orquestadores ayuda a simplificar la creación de servicios individuales, lo cual como hemos visto, ofrece muchas ventajas, pero también genera complejidades adicionales o mayores, como son enfrentarse a:

- **Seguridad.**

Dentro de una aplicación monolítica, todas las llamadas de función a función son seguras.

En los microservicios, hay que tener en cuenta que cada uno de ellos se debe autenticar, autorizar, encriptar y que se comunican entre sí.

También son necesarias herramientas de auditoría para el seguimiento de la comunicación servicio a servicio.

- **Resiliencia de la red.**

La resiliencia de la red se define como qué tan rápido puede volver a funcionar "algo" con todas sus capacidades y velocidad tras una caída/interrupción en la señal y reanudar la conexión.

Esta característica es imprescindible a la hora de prestar cualquier servicio a través de la red, y en el uso de microservicios, se debe lograr que cada uno de ellos sea resiliente, es decir, que, en caso de error, un microservicio puede reiniciarse en otra máquina para seguir estando disponible, para ello es necesario tener en cuenta su tolerancia a fallos, conmutación por error (failover), recuperación de desastres, interrupción de circuitos, aislamiento, etc.

La arquitectura de microservicios debe proporcionar alta disponibilidad.

- **Política de comunicación.**

En aplicación monolítica, se crea un único perímetro sólido de comunicación, pero en las arquitecturas de microservicios, se pueden generar cuellos de botella en algunos servicios, y además pueden ser dependencias para otros servicios.

Son necesarias unas políticas concretas para administrar todos los servicios, debe evitarse que se produzcan peticiones fraudulentas en un servicio, que realiza demasiadas llamadas y sobrecarga a los servicios a los que les está realizando solicitud.

Hay que administrar las cuotas y los límites de frecuencia de todos los servicios, controlar los servicios de forma eficaz, creando políticas que indiquen qué servicios pueden realizar llamadas y cuáles no.



- **Observabilidad.**

La observabilidad es más importante en las arquitecturas de microservicios que en las monolíticas donde los archivos de registro son suficientes para identificar el origen de un problema.

Con los microservicios, como varios servicios pueden abarcar una sola solicitud, la latencia, los errores y las fallas pueden ocurrir en cualquier servicio, por lo que se necesitan métricas, registros... que ayuden a los desarrolladores a identificar un problema.

Estos retos son importantes ya que si una aplicación se divide en microservicios, estos pueden ser decenas, cientos o miles, y deben ser escalables, y se comunicaran entre sí. Por tanto, es necesario realizar estos microservicios de forma correcta.

El uso de la malla de servicios proporciona soluciones de gestión a estos retos.

Categorías de desafío ante el uso de Microservicios

John Frizelle, arquitecto de plataforma para Red Hat Mobile, expuso estas ocho categorías de desafío en su charla de 2017 en Red Hat Summit:

- **Construcción:**

Es necesario invertir el tiempo necesario en identificar dependencias entre los servicios, así como tener en cuenta los efectos que el uso de los microservicios tiene sobre los datos.

- **Pruebas:**

En función de cómo se diseñen los servicios para apoyarse mutuamente, una falla en una parte de la arquitectura podría causar que falle un par de saltos, por lo que las pruebas de integración y de extremo a extremo cobran mucha más importancia en este tipo de arquitectura, y mayor dificultad.

- **Control de versiones:**

Cuando se actualiza a nuevas versiones se debe mantener la compatibilidad con versiones anteriores.

Se podrían soportar múltiples versiones en vivo para diferentes clientes, pero eso aumenta mucho la complejidad en mantenimiento y administración.

- **Implementación:**

Hay que plantear bien como se a realizar la implementación de los servicios y en qué orden ya que es necesario invertir en mucha automatización.

- **Registro:**

Se necesitan registros centralizados para unir todo, y poder realizar una gestión correcta.



- **Monitoreo:**

Tener una vista centralizada del sistema es imprescindible para poder identificar las fuentes de problemas.

- **Depuración:**

Al tratarse de un número de servicios que puede ser muy elevado, decenas, cientos o miles, no funcionará la depuración remota a través de su entorno de desarrollo integrado local (IDE) por lo que no es una opción.

- **Conectividad:**

Los servicios están conectados entre sí, con lo que ello implica.

6.2. Implementación de una arquitectura de microservicios

Aunque cada microservicio es independiente, se mantienen aislados unos de otros, pueden tener una estructura diferente, y se ejecutan en su propio entorno, todos deben encajar, han de contener puntos de conexión comunes, ya que solo se comunican entre sí a través de interfaces.

Cada microservicio debe diseñarse de la forma más simple posible para que la conexión tenga poco impacto en el proceso en sí. Para ello muchos desarrolladores confían en APIs REST, (cada microservicio se puede comunicar fácilmente con los demás e intercambiar información necesaria, por medio de HTTP, como GET o POST).

Para lograr ese aislamiento entre ellos que es lo que proporciona las ventajas de los microservicios, se pueden utilizar diferentes opciones:

- **Basarse en Contenedores:**

Esta es la forma más común de desarrollar una arquitectura de microservicios.

Los contenedores no utilizan máquinas virtuales completas, sino que se parte de un mismo sistema operativo y se utiliza su núcleo o kernel, por lo que representan un método muy ligero de virtualización.

En los contenedores, los microservicios son completamente autónomos, pues todo lo que necesitan para funcionar está ya contenido en ellos.

- **Máquinas virtuales:**

Se puede crear una máquina virtual para cada microservicio, lo que conlleva que cada máquina necesita su propio sistema operativo y por tanto muchos recursos.

Lógicamente, de esta forma, también se logra que cada microservicio funcione de forma aislada del resto.



- **Instalar una instancia-servidor física propia para cada microservicio.**

Esta opción, no es práctica, ya que resultaría en un derroche de recursos, por lo que suele optarse por la virtualización.

Hay que lograr un aislamiento real entre los microservicios, por ello no se recomienda ejecutar varios microservicios en un único servidor ni tampoco todos juntos en un contenedor, que podría provocar conflictos entre las diferentes aplicaciones.



+ Info

Hay que evitar sobrecargas en el sistema.

Se utilizan balanceadores de carga que reparten la carga automáticamente entre las diferentes instancias para evitar fallos.

6.3. Ejemplos de sistemas con arquitectura de microservicios

Muchas grandes empresas con sistemas monolíticos consolidados, ya han decidido cambiar a microservicios, lo que les ha ayudado también a resolver ciertos problemas u optimizar sus procesos.

Algunos ejemplos son Netflix, Spotify y eBay, que vamos a indicar a continuación (compañías como Google o Amazon también trabajan con microservicios):

6.3.1. Netflix

Cuando únicamente enviaba por correo películas en formato DVD, (no era todavía un servicio de streaming) se basaba en un sistema monolítico.

En 2008 un error en una base de datos provocó una interrupción del servicio durante cuatro días, y fue a partir de ese momento cuando la empresa decidió pasar a la división de su sistema en microservicios, logrando de este modo también lanzar los cambios con mayor rapidez.

En el pasado, cuando todavía no era un servicio de streaming, sino que enviaba por correo películas en formato DVD, Netflix se basaba, como la mayoría de empresas, en un sistema monolítico, hasta que en 2008 un error en una base de datos provocó una interrupción del servicio durante cuatro días. A partir de este momento se decidió desintegrar el antiguo sistema y dividirlo en microservicios. Con esto se logró que la empresa pudiera realizar las reparaciones y los cambios mucho más rápidamente.



Dada la gran extensión del sistema de Netflix, la empresa desarrolló un programa propio, que es conocido como Conductor, para poder coordinar los diferentes microservicios entre sí, que permite gestionar los microservicios de forma central (pausar o reiniciar) o escalarlos. En el núcleo de este programa trabaja un servicio que puede planificar los procesos de forma automatizada llamado Decider, y que también reacciona a eventos en el workflow (flujo de trabajo).

Netflix también ha desarrollado otros programas para trabajar con microservicios eficazmente, son Dynamite (datastore), Mantis (Stream processing), y Vizceral (traffic intuition).



+ Info

Con frecuencia Netflix recorre a programas de código abierto, y publica sus programaciones en la red. Puedes consultarse en su perfil en GitHub.

<https://github.com/Netflix>

6.3.2. Spotify

El mercado de audio de streaming tiene una gran competencia actualmente. Spotify debe competir con grandes empresas como Google, Amazon y Apple.

Spotify, apuesta por los microservicios, para mejorar la prestación de su servicio frente a la competencia publicando rápidamente sus propias innovaciones, y también respondiendo ante las innovaciones que pueda realizar la competencia.

También ante el constante crecimiento de suscriptores, sus desarrolladores deben cubrir ese incremento y observar cosas como los derechos de licencia.

Los microservicios ofrecen a Spotify la solución adecuada sus necesidades.

Spotify tiene más de 800 microservicios activos, y para una gran parte de ellos se utiliza Java, con el fin de mantener el flujo de trabajo, aunque se podrían utilizar diferentes lenguajes, como los programadores cambian de equipo constantemente, el trabajo resulta más viable si utilizan todos el mismo lenguaje.

6.3.3. eBay

eBay con su progreso, llegó a acumular 3,4 millones de líneas de código en un único archivo, lo que produjo que la empresa cambiara su sistema monolito en microservicios en Java, (se comunican mediante REST).



+ Info

En el inicio de un proyecto online con pocos usuarios activos, una aplicación monolítica es suficiente, pero si se realiza un gran crecimiento el sistema resultara torpe y estático, dificultando el crecimiento. Por ello hay que plantear correctamente la arquitectura adecuada a la hora de realizar un desarrollo.

7. El uso de contenedores (Kubernetes y Docker)

Para el desarrollo de una arquitectura de microservicios, es necesario el uso de los contenedores (bins en inglés) y los orquestadores.

Veamos diferentes formas de describir la función, ventajas y capacidades de los contenedores:

- Son los que permiten empaquetar todo lo necesario para que un servicio se ejecute de manera encapsulada, completamente independiente del servidor anfitrión.
- Son los más adecuados para ofrecer microservicios.

Proporcionan entornos virtuales portátiles y aislados para que las aplicaciones se ejecuten sin interferencia de otras aplicaciones en ejecución.

- Reúnen todos los recursos necesarios para que una aplicación necesita sea ejecutada con éxito, y que si se transporta a otra máquina siga funcionando correctamente.
- Se maximiza la portabilidad, los contenedores pueden ejecutarse en varias plataformas diferentes en la nube.
- Ofrecen aplicaciones escalables de alto rendimiento, en cualquier infraestructura que se elija.

Por tanto, aseguran la disponibilidad del servicio independientemente del sistema operativo en el que este alojado.

La principal finalidad de los contenedores es fomentar la ligereza y portabilidad de una aplicación para pueda transferirse entre diferentes entornos en el menor tiempo posible. Una aplicación se puede dividir en muchos dominios, todos los cuales residen en contenedores.



7.1. Microservicios y contenedores

Los Microservicios son aplicaciones ligeras, que pueden desarrollarse utilizando diferentes lenguajes de programación, con sus propias dependencias, bibliotecas y requisitos.

Es necesario empaquetar la aplicación junto con sus dependencias para asegurar que tenga todo lo necesario para su correcta ejecución.

Los Contenedores encapsulan Microservicios y sus Dependencias, pero no los ejecutan directamente, lo que hacen los contenedores es ejecutar las Imágenes de Contenedores.

Una Imagen de Contenedor:

- Agrupa la aplicación junto sus dependencias y su ambiente de ejecución.
- Entonces se implementa un **Contenedor** a partir de la **Imagen de Contenedor** que ofrece un entorno ejecutable aislado para la aplicación.

Los Contenedores se pueden implementar desde una imagen específica en muchas plataformas, como:

- Máquinas virtuales.
- Estaciones de trabajo.
- Nube pública.
- Etc.

Existen diferentes soluciones en el mercado para realizar la función de contenedores, indicaremos algunas más adelante. La más extendida es Docker, pero su uso no es satisfactorio si la arquitectura de microservicios requiere que haya cientos de contenedores alojados en diferentes hosts. Ante esta situación ha cobrado auge el uso de Kubernetes.

Docker se ejecuta en un nodo único, y Kubernetes se diseñó para ejecutarse en un clúster.

Docker y Kubernetes son tecnologías distintas que funcionan bien de forma conjunta para compilar, entregar y escalar aplicaciones en contenedores.

A modo de resumen:

- **Dockers.**
 - Es un runtime (sistema operativo) para contenedores.
 - Docker son los contenedores, el que tiene la información necesaria para desarrollar una aplicación.
 - Docker lidera el mercado de contenedores, pero existen otras alternativas.

(La empresa CoreOS tiene su propio estándar de contenedor llamado Rocket, y hay muchos productos y estándares que se están construyendo alrededor de esta tecnología).



- **Kubernetes.**
 - Es un gestor de contenedores, "el orquestador", que coordina a todos los contenedores.
 - Es el estándar de facto para gestionar los contenedores, ya sean Docker u otros.

7.2. Orquestación de contenedores

Con las **Imágenes de Contenedor**, podemos limitar el código de la aplicación, su tiempo de ejecución y todas sus dependencias en un formato predefinido.

Con tiempos de ejecución de contenedores como, por ejemplo, containerd, runC, o rkt, se pueden usar esas imágenes preempaquetadas para crear uno o más Contenedores.

Estos entornos de ejecución son buenos para ejecutar contenedores en un solo "host", pero para poder tener una solución escalable y tolerante a fallas, creando entornos de control de calidad (QA) y producción (Prod), dicha opción ya no es viable porque las aplicaciones y los servicios deben cumplir mayores requisitos, necesitamos, después de conectar varios nodos juntos, crear un solo "controlador/unidad de administración", que se conoce generalmente como un "Orquestador de Contenedores".

Los "Orquestadores de Contenedores" son herramientas que agrupan sistemas para formar Clústeres.

En estos clústeres se habilita un ambiente para la automatización y la escalabilidad en la implementación y administración de Contenedores, pudiendo así cumplir los siguientes requisitos necesarios:

- Uso óptimo de recursos.
- Tolerancia a fallos.
- Escalabilidad bajo demanda.
- Descubrimiento automático para comunicarse entre sí automáticamente.
- Accesibilidad desde el mundo exterior.
- Actualizaciones/reversiones sin interrupciones y/o tiempo de inactividad.

Como ya hemos indicado, la orquestación de contenedores facilita la administración de los contenedores, que es imprescindible cuando se trata de un número elevado de ellos que se ejecutan en una infraestructura global (se puede mantener manualmente unos cuantos contenedores o escribir scripts, pero es inviable para cientos o miles de contenedores). Para facilitar esta administración, en general, todos los Orquestadores de Contenedores pueden:

- Agrupar "hosts" mientras se crea un clúster.
- Programar contenedores para que se ejecuten en "hosts" en el clúster según sea la disponibilidad de recursos.



- Permitir la comunicación entre sí de los contenedores de un clúster con independencia del host en el que estén implementados en el clúster.
- Vincular los contenedores y los recursos de almacenamiento.
- Agrupar conjuntos de contenedores similares y vincularlos a construcciones de balanceo de carga.

Así se simplificar el acceso a las aplicaciones en contenedores creando un nivel de abstracción entre los contenedores y el usuario.

- Gestionar el uso de recursos para optimizarlos.
- Permitir la implementación de políticas.

Es necesario proteger el acceso a cada contenedor.



+ Info

Casi todos los Orquestadores de Contenedores se pueden implementar en diferentes infraestructuras:

- Servidores físicos.
- Máquinas virtuales.
- Propias instalaciones (On-premise).
- La Nube Privada o en la Nube Pública.

7.3. Seguridad en los contenedores

Al trabajar en entornos de infraestructura compartida, la seguridad se debe trabajar de un modo diferente, es necesario ya en su proceso de desarrollo, como parte del ciclo de vida de entrega continua para lograr la reducción del riesgo y de las vulnerabilidades, de dos formas:

- De forma automatizada para eliminar los puntos de contacto manuales.
- De forma y extendida en el mantenimiento y la operación de la infraestructura subyacente.



Para ello se debe trabajar correctamente:

- La protección de sus imágenes de contenedor en la canalización de desarrollo.
- De la plataforma del host en tiempo de ejecución.
- Y las capas de aplicación.

Para proteger los contenedores, los principales aspectos a tener en cuenta son:

- **La seguridad del host del contenedor.**

Esta protección comienza con la elección del sistema operativo, debe ser un sistema operativo distribuido que esté optimizado para ejecutar contenedores.

Hay que deshabilitar o eliminar los servicios innecesarios fortaleciendo así el sistema operativo. Añadir una capa de seguridad y herramientas de supervisión para garantizar que el host se ejecute correctamente.

Resulta útil el uso de herramientas del control de aplicaciones.

- **La seguridad en el contenedor y el tráfico de red del contenedor.**

Cuando el contenedor se está ejecutando, interactúa con otros contenedores y recursos, por lo que hay que supervisar todo ese tráfico de red garantizando que pasa a través de un IPS ((Intrusion Prevention System) o sistema de prevención de intrusiones).

Hay que implementar el IPS en cada host, para supervisar todo el tráfico de forma efectiva sin repercutir significativamente en el rendimiento.

Es fundamental el uso de controles anti-malware en tiempo real, que se ejecutan en el contenido del contenedor funcionen a la perfección, ya que cuando el contenedor se está ejecutando, constantemente está procesando datos, generando archivos de caché y de registro, etc.

El uso de un IPS, desempeña también un papel importante aquí, utilizando un patrón llamado parche virtual:

- El motor del IPS, si una vulnerabilidad queda expuesta de forma remota, puede detectar intentos de exploit y lanzar paquetes para proteger la aplicación.
- En lugar de forzar una solución de emergencia, proporciona un tiempo para poder abordar la causa raíz en la siguiente versión del contenedor.

- **El comportamiento malicioso en su aplicación.**

También es de ayuda a la hora de implementar una aplicación en un contenedor, el uso de un control de seguridad de autoprotección e en tiempo de ejecución (RASP), ya que:

- Se ejecutan en el código de aplicación y pueden interceptar o enlazar llamadas claves en su código.



- Ofrecen la supervisión del lenguaje de consulta estructurado (SQL), la remediación y comprobación de dependencias, la verificación de URL y demás controles.
- También puede realizar la identificación de la causa raíz.

Como se colocan en el código de la aplicación, estos controles de seguridad pueden proporcionar ayuda para conectar los puntos entre una incidencia de seguridad y la línea de código que lo creó.

- **Capas de administración.**

La tecnología de agrupación, programación y orquestación, herramientas pueden administrar grupos de contenedores utilizando una capa de administración de contenedores bien definida, que proporciona que los contenedores puedan escalar y ser resistentes.

Para construir con éxito aplicaciones "contenerizadas" son necesarias estas capas de administración.

7.4. Los contenedores en La Nube

La portabilidad ha sido un gran empuje en el desarrollo de la Computación en La Nube, esta tecnología permite abstraer las aplicaciones en contenedores virtuales que se pueden mover de una nube a otra.

Ahora hay una forma estándar de dividir las aplicaciones en objetos distribuidos o contenedores, y es más fácil crear sistemas tolerantes a fallos, así como la gestión de la carga de trabajo. El uso de la agrupación y la orquestación, garantiza que las aplicaciones que existen dentro de los contenedores puedan escalar y ser resistentes. Además de poder ubicar las aplicaciones en diferentes máquinas físicas y/o virtuales, se puede hacer también en La Nube Privada, La Nube Pública o La Nube Híbrida.

El hecho de que grandes empresas como AWS, HP o IBM confíen en el uso de contenedores da lugar a que exista un soporte directamente desde las herramientas y tecnología empresariales existentes, y que también aparezcan numerosas startups (empresas emergentes) bien financiadas, que ofrezcan soluciones innovadoras para aumentar el interés del desarrollo de contenedores, y que su uso sea cada vez más productivo.

8. Malla de servicios (Service Mesh)

Cuando se realiza una aplicación con microservicios, se facilita la seguridad, la resiliencia de la red, la política y la observabilidad, pero hay que tener en cuenta que el número de estos microservicios pueden ser muy elevado, y que deben ser escalables. Esta escalabilidad debe hacerse de forma correcta.

Se deben crear aplicaciones sólidas que estarán compuestas por muchos microservicios en una infraestructura seleccionada, para ello es necesario el uso de una malla de servicios.



La malla de servicios es la capa de infraestructura configurable, que proporciona la comunicación entre cada instancia de servicio, de forma fluida, confiable y rápida.

La malla de servicios (Service Mesh) es la capa de comunicación en su configuración de microservicio.

- Todas las solicitudes desde y hacia cada uno de tus servicios pasarán por la malla.
- Cada servicio tendrá su propio servicio de proxy y todos estos servicios de proxy juntos forman la "malla de servicios".
- Si un servicio quiere llamar a otro servicio, no llama directamente al servicio de destino, enruta la solicitud primero al proxy local y el proxy la enruta al servicio de destino.

(Su instancia de servicio no tiene ninguna idea sobre el mundo exterior y solo conoce el proxy local).



+ Info

Cuando habla de mallas de servicios, se utiliza siempre el término **"Sidecar"**, que se refiere a un proxy que está disponible para cada instancia de su servicio, cada "Sidecar" se encarga de una instancia de un servicio.

Your Service (servicio) → Envoy (solicitud enviada).

El objetivo de las mallas de servicios es proporcionar funcionalidades específicas para administrar y controlar las relaciones de comunicación entre los servicios.

Las mallas de servicios proporcionan:

- Descubrimiento de servicios.
- Observabilidad (métricas).
- Limitación de tasa.
- Rotura de circuito.
- Cambio de tráfico.



- Balanceo de carga.
- Autenticación y autorización.
- Seguimiento distribuido.

Las mallas de servicios:

- Son transparentes para la aplicación.
- Con las herramientas que ofrecen, permiten excluir de la ejecución de un determinado servicio, cosas tales como la supervisión, las redes y la seguridad.
- Supervisa todo el tráfico a través de un proxy.

El proxy se implementa mediante un patrón de proxy de sidecar en los microservicios, que separa la lógica de la aplicación de las funciones de red.

- Posibilitan que los equipos de operaciones y de desarrollo separen el trabajo entre sí.

Los operadores y los desarrolladores de servicios pueden centrarse en crear y administrar aplicaciones.

8.1. Arquitectura

Básicamente, una malla de servicios consiste en servicios y proxies que se ejecutan como archivos adicionales a los servicios.

Todas las solicitudes hacia o desde un servicio pasan por dos proxies dentro de la malla: el proxy para el servicio de llamadas y el proxy para el servicio receptor.

También incluye cierta autoridad que configura esos proxies para combinar en un sistema distribuido adecuado:

- **Data Plane. El plano de datos de la malla de servicios:**

Administra los proxies y servicios.

Es responsable de:

- Registro de servicio:

El plano de control debe tener una lista de servicios y extremos disponibles para proporcionarlos a los proxies.



El plano de control compila este registro mediante una consulta al sistema de programación de infraestructura subyacente, (como por ejemplo Kubernetes), para obtener una lista de todos los servicios disponibles.

- Configuración del proxy de sidecar:

Incluye las políticas y la configuración en toda la malla que los proxies deben tener en cuenta para realizar sus funciones adecuadamente

- **Control Plane. El plano de control de la malla de servicios:**

Es la autoridad que proporciona la política y la configuración al plano de datos.

Permite que los proxies realicen las siguientes funciones:

- Descubrimiento de servicios.
- Enrutamiento del servicio.
- Balanceo de cargas.
- Autenticación y autorización.
- Observabilidad.



+ Info

La configuración del Envoy (solicitud enviada) consiste principalmente en:

- Oyentes.
- Rutas.
- Clusters.
- Puntos finales.



8.2. Características

Algunas de las funcionalidades de las mallas de servicios para administrar y controlar las relaciones de comunicación entre los servicios son:

8.2.1. Multiusuario

El patrón de implementación de multiusuario aísla grupos de microservicios entre sí.

La forma más simple de multiusuario es tener infraestructura dedicada a una sola instancia, donde cada usuario, sin compartir infraestructura, tiene su propia red, almacenamiento, procesamiento y componentes adicionales (como Kubernetes y microservicios), pero esta forma de multiusuario en muchas situaciones ofrece una infraestructura ineficiente, por tanto, es más eficaz compartir la infraestructura entre los usuarios, y que la malla de servicios se encargue de separarlas.

Los multiusuarios de malla de servicio se basan en uno de los siguientes formatos:

- **Usuarios del espacio de nombres.**

El formulario de usuario del espacio de nombres proporciona a cada instancia un espacio de nombres dedicado dentro de un clúster, y como cada clúster admite varios usuarios, el usuario del Espacio de nombres maximiza el uso compartido de la infraestructura.

Para restringir la comunicación entre los servicios de diferentes usuarios, expone solo un subconjunto de servicios fuera del espacio de nombres (con una configuración de archivo adicional) y usa las políticas de autorización de malla de servicios para controlar los servicios expuestos. Configura cada espacio de nombres de forma individual para el conjunto de servicios disponibles.

Debido a que el acceso a cada servicio está autorizado, solo los usuarios permitidos pueden acceder a los servicios de los otros. Si bien la federación de varias mallas admite este caso de uso, no es necesario crear una federación de malla múltiple.

Un espacio de nombres:

- Puede abarcar uno o más clústeres.
- Define al usuario de forma exclusiva.
- Es independiente de los clústeres que lo admiten.

Dos mallas de servicios diferentes pueden tener el mismo espacio de nombres.

Un ejemplo de este concepto es una malla de servicios que representa una instancia de etapa de pruebas y una malla de servicios que representa una instancia de producción. Ambos pueden tener un espacio de nombres de cliente. Como este esquema de nombres es confuso, no es lo ideal.



- **Usuarios de clústeres.**

El formulario de usuario del clúster dedica exclusivamente un clúster completo, incluidos todos los espacios de nombres, a un usuario.

Una instancia también puede tener más de un clúster.

Cada clúster tiene su propia malla.

8.2.2. Seguridad

La seguridad es imprescindible en cualquier aspecto informático.

En la arquitectura de microservicios surgen unas necesidades de seguridad adicionales en comparación con otras arquitecturas, que deben ser abordadas por las mallas de servicios, como son: la autenticación, la autorización y el control de flujo de tráfico entre microservicios.

En una red tradicional, la seguridad se basa en un perímetro sólido para evitar un acceso no autorizado, y una vez salvado ese perímetro, los usuarios ya son considerados actores de confianza y se les permite comunicarse sin volver a verificar su identidad. Sin embargo, en la arquitectura de microservicios, se considera un entorno de confianza cero.

La malla de servicios facilita alcanzar la confianza cero, ya que proporcionan estas identidades de autenticación y autorización a través de una autoridad certificadora central que proporciona certificados para cada servicio. Usan estas identidades para autenticar y autorizar servicios dentro y fuera de la malla.



+ Info

El concepto de confianza cero, popularizado por Forrester en 2010, es un entorno donde ya no se supone que nada dentro de un perímetro de seguridad específico sea confiable. En cambio, se supone que la red está comprometida y no está al tanto.

Todo se verifica. En este caso, el único perímetro de confianza se encuentra dentro del servicio. Cualquier otro elemento, incluso si está dentro de la misma red, no es de confianza implícita.

Los desarrolladores, mediante las autoridades certificadoras y la disponibilidad de certificados, pueden implementar políticas de autorización que proporcionen un control detallado sobre los servicios que pueden comunicarse entre sí, y también es posible que especifiquen detalladamente las rutas de acceso y los verbos HTTP que se permiten para ciertos servicios.



+ Info

Un ataque de intermediario es un ciberataque en el que el atacante transmite en secreto y posiblemente altera las comunicaciones entre dos partes que creen que se están comunicando directamente entre sí.

Una vez implementada la malla de servicios, es ella la responsable de la encriptación y la descryptación de todas las solicitudes y respuestas

8.2.3. Observabilidad y análisis

La observabilidad es un conjunto de actividades que incluyen la medición, la recopilación y el análisis de varias señales de un sistema.

En las arquitecturas de microservicio, la observabilidad es más compleja, ya que las solicitudes no llegan a un solo servicio, los datos de respuesta se deben recopilar a partir de varios servicios para obtener la respuesta completa.

Ante esta complejidad, todo el tráfico en la malla pasa por un proxy, ya sea hacia o desde un servicio, y así este proxy informa de la solicitud, y se produce la misma vista integral que si se tratara de una aplicación monolítica.

Generalmente, para proveer observabilidad, una malla de servicios genera distintos tipos de telemetría (monitorización y al análisis de información sobre sistemas informáticos para monitorizar el rendimiento e identificar problemas) como son métricas, seguimientos distribuidos y registros de acceso.

- **Métricas.**

La malla produce métricas para todo el tráfico que ingresa a la malla, tanto dentro de ella como fuera de ella, para ayudar a los desarrolladores a observar y comprender el comportamiento del servicio.

Estas métricas son, por ejemplo, la cantidad de solicitudes por segundo y los tiempos de respuesta de las solicitudes y también las tasas de errores.



La malla puede producir las siguientes métricas:

- **Métricas a nivel de proxy:**

Los proxies de sidecar generan un gran conjunto de métricas sobre todo el tráfico del proxy de entrada y de salida, incluyendo estadísticas detalladas sobre las funciones administrativas del proxy (por ejemplo, la información de configuración y la información de estado).

- **Métricas de nivel de servicio:**

Abarcan las cuatro señales importantísimas de la supervisión:

- » Latencia.
- » Tráfico.
- » Errores.
- » Saturación.

- **Métricas del plano de control:**

Supervisan el plano de control de la malla de servicios en lugar de los servicios dentro de la malla.

- **Generar intervalos de seguimientos distribuidos.**

Estos seguimientos se utilizan para seguir una sola solicitud a través de la malla en varios servicios y proxies.

- **Generar registros de acceso.**

Para poder realizar la auditoría a nivel del servicio, la malla puede generar un registro de acceso completo, que abarca todas las llamadas de servicio (fuente de la llamada y su destino).



+ Info

Istio, plataforma de red de servicios con tecnología de open source, que permite controlar el intercambio de datos entre los microservicios, ofrece información sobre la observabilidad en su web oficial.

<https://istio.io/latest/docs/concepts/observability>.



8.2.4. Cumplimiento de políticas y reglas

Existen unas políticas y reglas impuestas por regulaciones del gobierno o la industria o bien autoimpuestas por la propia empresa desarrolladora.

Para cumplirlas es necesario:

- **Supervisar y auditar las cargas de trabajo**, viendo si se produce algún incumplimiento
Supervisión y auditoría: Las cargas de trabajo de supervisión y auditoría ayudan a determinar si hay incumplimientos de políticas o reglas dentro del sistema.

- **Controlar la seguridad** (sistemas seguros).

Todos los sistemas deben ser autenticados, y proporcionar control acceso autorizado a todos sus extremos.

- **Redundancia:**

Debe realizarse la implementación de cada servicio en más de una ubicación, para evitar un punto único de fallo.

Si se implementa en la misma zona la arquitectura no proporciona alta disponibilidad.

- **Alta disponibilidad:**

Para ofrecer una implementación de alta disponibilidad, cada servicio y cada componente deben implementarse como mínimo en dos zonas.

(Un componente de servicio configura una implementación de servicio).

Así se puede garantizar la continuidad, ya que una si ocurre una interrupción en una zona, la implementación de alta disponibilidad debe continuar funcionando.

La zona no se convierte en un punto único de fallo.

La propia funcionalidad del microservicio puede reaccionar a una interrupción de zona, la configuración de la malla de servicios permite, además, que se analice de forma automática la redundancia completa en al menos dos zonas.

- **Recuperación ante desastres:**

A diferencia del concepto de alta disponibilidad, la recuperación ante desastres hace que un sistema continúe funcionando durante una interrupción en una sola región.

La malla de servicios puede analizar de forma automática que la implementación se realice de forma adecuada para garantizar que un sistema continúe funcionando ante este tipo de interrupción.



- **Partición (multiusuario):**

Los microservicios permiten implementar sistemas de multiusuario.

La configuración de la malla de servicios debe ayudar a garantizar que se realice la partición entre usuarios de forma correcta.

- **Propiedades de entorno de ejecución:**

Las políticas y reglas pueden ser:

- Enfocadas en una implementación o configuración estática.
- O pueden ser políticas del entorno de ejecución.

Por ejemplo, una política del entorno de ejecución puede aplicar un límite de latencia superior.

(Aquí cobra importancia la función de resiliencia, que indicamos un poco más adelante).



+ Info

Algunas políticas y reglas no se pueden aplicar mediante una malla de servicios, como almacenar durante una cantidad determinada de años, todos los datos y su historial, o controlar que los datos de usuario estén en el mismo país que la ubicación de inicio especificada.

8.2.5. Control del tráfico

Una malla de servicios permite varios controles en cuanto al tráfico que se produce:

- **Controla el flujo de tráfico entre servicios tanto hacia la malla como hacia servicios externos.**

Los recursos personalizados, que variaran según la malla elegida, permiten a los usuarios administrar este tráfico, pudiendo:

- **Crear lanzamientos de versiones Canary.**

Las versiones Canary generalmente se lanzan primero, o un pequeño porcentaje, como el 2% de los servidores, principalmente para la verificación del tráfico, también conocidas como pruebas canarias o pruebas en escala de grises.



Su nombre deriva del uso de un pájaro (canario) para comprobar si existían gases tóxicos en las minas.

Las pruebas canarias simples generalmente se verifican mediante pruebas manuales, mientras que las pruebas canarias complejas requieren una infraestructura de monitoreo relativamente completa.

Si pasan las pruebas las versiones se actualizan y si fallan, se revierte directamente la versión y la liberación falla.

- **Crear lanzamientos azul-verde.**

Es un modelo de lanzamiento de aplicaciones que transfiere poco a poco el tráfico de usuarios de cierta versión anterior de una aplicación o microservicio a una versión nueva casi idéntica, cuando ambas se encuentran en producción.

La versión anterior se denomina entorno azul, mientras que la versión nueva se conoce como entorno verde.

Cuando el tráfico de producción se transfiere por completo del entorno azul al verde, la versión azul puede conservarse en caso de ser necesaria una restauración, o extraerse de la producción y actualizarse para convertirse en la plantilla a partir de la cual se realizará la próxima actualización.

- **Y control detallado sobre rutas específicas para servicios.**

- **La malla de servicio:**

- Mantiene un registro de todos los servicios en la malla por nombre y por sus extremos respectivos.
- Mantiene el registro para administrar el flujo de tráfico.

(Por ejemplo, las direcciones IP del Pod de Kubernetes).

Cuando se usa este registro de servicio y se ejecutan los proxies junto con los servicios, la malla puede dirigir el tráfico al extremo adecuado.

- **Balanceo de cargas.**

Con los microservicios, hay varias instancias de cada servicio en ejecución (por ejemplo, pods en Kubernetes), y las cargas del tráfico se balancean en todas las instancias.

Esto puede ser controlado por la malla de servicios, normalmente el comportamiento en las instancias del servicio es round-robin, pero puede ser aleatorio, en función de un porcentaje específico de tráfico, o dirigido al servicio con el menor tráfico.

El equilibrio de carga funciona de abajo hacia arriba, se puede seleccionar el servicio "menos ocupado", que es el equilibrio de carga en un nivel alto.



El DNS round-Robin es una técnica de distribución de carga, equilibrio de carga o tolerancia a fallas que proporciona múltiples hosts de servicios de protocolo de Internet redundantes, por ejemplo, servidor web, servidores FTP, mediante la administración de las respuestas del Sistema de nombres de dominio (DNS) a las solicitudes de direcciones. desde los equipos cliente de acuerdo con un modelo estadístico apropiado.

8.2.6. Resiliencia

Una malla de servicios puede aumentar la resiliencia de invocación de los microservicios. Existen dos clases de medidas de resiliencia:

- **Aumentar la confiabilidad de las invocaciones de microservicios.**

La confiabilidad de una invocación de microservicio aumenta si las fallas se abstraen del emisor. Si se produce un error, la malla de servicios puede usar las siguientes estrategias para tratar de abordarlo con transparencia sin mostrar una falla al emisor:

- **Tiempo de espera.**

Cantidad de tiempo que un proxy Envoy debe esperar las respuestas de un servicio determinado, lo que garantiza que los servicios no se queden esperando respuestas indefinidamente y que las llamadas tengan éxito o fallen dentro de un período de tiempo predecible.

- **Reintentar.**

Especifica el número máximo de veces que un proxy Envoy intenta conectarse a un servicio si falla la llamada inicial.

Los reintentos pueden mejorar la disponibilidad del servicio y el rendimiento de la aplicación al asegurarse de que las llamadas no fallen de forma permanente debido a problemas transitorios, como un servicio o una red sobrecargados temporalmente.

El intervalo entre reintentos (25ms +) es variable.

El comportamiento de reintento predeterminado para las solicitudes HTTP es reintentar dos veces antes de devolver el error.

- **Disyuntores.**

La rotura de circuitos es un patrón importante para crear aplicaciones de microservicio resistentes. Hay que configurar la interrupción del circuito para conexiones, solicitudes y detección de valores atípicos.

La interrupción de circuitos permite escribir aplicaciones que limitan el impacto de fallas, picos de latencia y otros efectos indeseables de las peculiaridades de la red.



Hay que configurar las reglas de interrupción del circuito y luego probar la configuración "disparando" intencionalmente el interruptor.

(El término disyuntor significa un aparato eléctrico que abre automáticamente el paso de la corriente eléctrica).

- **Crear fallas de invocación de forma intencional.**

La malla de servicios permite incorporar fallas de invocación intencional para probar que una aplicación funcione de forma correcta, estas fallas pueden ser:

- **Una demora o retraso.**

La invocación se retrasa de forma intencional y esa demora prueba la capacidad de la aplicación para lidiar con una variación en latencias.

- **La anulación o aborto.**

Una anulación interrumpe la invocación, la aplicación detecta una falla de invocación y decide cómo abordarla.

Por ejemplo, una aplicación puede invocar cuatro microservicios de forma secuencial para procesar la entrada y obtener un resultado. Si una de estas invocaciones falla, la aplicación puede volver a invocarla para ver si funcionan en el segundo intento.

Estas medidas se aplican en el entorno de ejecución a las invocaciones reales en un sistema de producción.

8.3. Consideraciones del diseño

Hay que tener en cuenta algunos aspectos sobre las mallas de servicios a la hora de considerarlas como una solución perfecta para el diseño de un sistema de microservicios y su implementación.

Estos aspectos son:

- **Sobrecarga de procesamiento.**

Hay que determinar si la sobrecarga de un caso de uso determinado es significativa, analizándola con mediciones del rendimiento y la escalabilidad.

Estos casos son:

- Las invocaciones de un microservicio a otro se enrutan a través de un proxy, y puede usarse también un balanceador de cargas.
- La realización de un seguimiento de las invocaciones y, posiblemente, se modifican a través de la encriptación.
- La encriptación, que, aunque no causa una sobrecarga significativa a nivel individual, en conjunto, sí que aumenta la latencia y los requisitos de recursos.



- **Complejidad del diseño de configuración.**

Crear una configuración de la malla de servicios es una actividad de diseño que debe garantizar que los requisitos se implementen de forma correcta.

Para ello es necesario tener conocimiento sobre las capacidades de configuración de las mallas de servicios en general, y sobre cómo crear la configuración correcta para aplicaciones específicas.

La configuración de una malla de servicios debe reflejar los requisitos del sistema.

- **Prueba la validez de la configuración.**

Hay que utilizar herramientas para validar la configuración de una malla después de su implementación (por ejemplo, Istioctl Analyze).

Como la configuración puede ir variando, es necesario también repetir esta validación de forma constante como parte del proceso de CI/CD.

La prueba la configuración de la malla de servicios debe mostrar el comportamiento expresado en la configuración.

- **Verifica la configuración de la malla de servicios.**

Aunque exista un plano de control de la malla de servicios, esto no garantiza de forma automática la seguridad y confiabilidad del sistema.

Debe probarse la configuración de una malla de servicios y verificar que se comporta de forma correcta, para evitar problemas como, por ejemplo, las invocaciones no seguras que no se detectan.

Hay que realizar una revisión de la configuración de la malla de servicios cuando se realiza algún cambio.



+ Info

Una malla de servicios no abarca todos los aspectos de seguridad que requieren implementación en una configuración empresarial, la malla de servicios aborda los aspectos relacionados con la comunicación del servicio, pero los requisitos de seguridad de la infraestructura, como firewalls y seguridad de red, deben ser tratados por separado.



8.4. Pruebas

Se deben realizar pruebas que garanticen la configuración y el funcionamiento adecuados de una malla de servicios, realizando las pruebas integrales que incluyan las siguientes verificaciones:

- **Verificación de configuración general de la malla de servicios.**
- **Comprobación de la configuración de la malla de servicios con los requisitos de microservicios:**
 - Comunicación de los microservicios.
 - Seguridad de la comunicación (uso de HTTP o HTTPS).
 - Comportamiento dinámico: La malla de servicios debe limitar la comunicación lo suficiente como para evitar sobrecarga en uno o más microservicios.

La configuración de la malla de servicios se realiza a través de archivos de configuración declarativos que forma parte de repositorio de código, contienen especificaciones de casos de uso, como por ejemplo que microservicios pueden comunicarse y cuáles no, o indicar cuánta capacidad de procesamiento es posible y cuándo se debe limitar.

Hay que realizar pruebas positivas y negativas, las que ofrezcan resultados positivos indican que la función o el comportamiento están presentes, y las negativas confirman que está ausente una funcionalidad o función específica. Estas pruebas podrán ser de unidad o de integración, según los casos de uso.

También se han de realizar pruebas positivas y negativas para probar la comunicación, por ejemplo, si deseamos que el MS1(microservicio1) llama al MS2, pero no al revés, hay que establecer una prueba para confirmar que MS1 puede llamar a MS2 y establecer otra para confirmar que MS2 no puede acceder a MS1.

Una forma de compilar las pruebas es implementar dos microservicios de prueba, cerrando uno para MS1 y otro para MS2, de forma que ambos puedan invocarse entre sí. También, se puede establecer una prueba en la que MS1 y MS2 puedan invocar al otro sin que esté presente la malla de servicios, de forma que cuando la malla de servicios está presente, la invocación de MS1 a MS2 debe funcionar, mientras que la invocación de MS2 a MS1 debe fallar.

Es aconsejable definir pruebas adicionales que consideren la resiliencia, verificando, por ejemplo, si la inserción de errores crea demoras de invocación.

- **Comprobación de la versión de implementación del plan de control de la malla de servicios.**

Es necesario verificar la versión de implementación del plano de control.

Se puede instalar y ejecutar dos versiones diferentes del plano de control de la malla de servicios al mismo tiempo, donde las pruebas deben establecer que la versión actual del plano de control y la versión más reciente del plano de control se comporten de la misma manera.



Las pruebas de integración y de unidad específicas deben comportarse de la misma manera, si no es así, hay una degradación del servicio que se debe evaluar, (esto es habitual en pruebas y CI/CD). Es un proceso importante para cambios en la configuración de la aplicación y cambios en la plataforma.

Es recomendable que las pruebas de integración y unidad se ejecuten de forma continua como parte de la canalización de CI/CD.

8.5. Ejemplos de malla de servicios

Veamos algunas destacadas mallas de servicios:

- **Istio.**

La arquitectura de Istio contiene:

- **Un plano de datos.**

Consta de proxies de Envoy que controlan la comunicación entre microservicios:

- » Tráfico entrante (llamado Ingress).
- » Tráfico saliente (llamado salida).
- » Y tráfico entre servicios (tráfico de malla).

Cada instancia de microservicio (contenedor o VM) tiene un proxy Envoy dedicado.

Los proxies de Envoy también recopilan métricas.

- **Un plano de control.**

Es la capa de administración para los proxies de Envoy, administra los proxies para que se produzca el enrutamiento de invocación correcto.

El objeto binario istiod es el núcleo del plano de control y proporciona descubrimiento de servicios, configuración y administración de certificados.

Istio admite los modelos de implementación con:

- » Uno o varios clústeres de Kubernetes.
- » Una o varias redes.
- » Uno o varios planos de controles.
- » Una o varias mallas.



- **Linkerd.**

Es de código abierto, con licencia bajo Apache v2, y es un proyecto graduado de Cloud Native Computing Foundation. Se desarrolla de forma abierta en la organización Linkerd GitHub.

Linkerd es una malla de servicio para Kubernetes.

- **Anthos Service Mesh.**

Es la malla de servicios completamente administrada de Google Cloud, ofrece a los desarrolladores una distribución de Istio probada y compatible con Anthos, permitiendo crear e implementar una malla de servicios en Google Cloud o en clústeres de Anthos alojados en VMware con una asistencia completa de Google.

9. Soluciones de administración de contenedores

Para decidir cuál es la herramienta más adecuada, debemos considerar que características ofrece dicho software, como son:

- Puesta en marcha automatizada y restauración automatizada.
- Supervisión de la seguridad.
- Escalamiento y flexibilidad.
- Capacidades de administración ofrecidas.
- Conectividad y Orquestadores.

Indicamos algunas de las herramientas más destacadas en el mercado:

9.1. Docker



Logo de docker. Fuente: Wikipedia

Docker es el sistema más utilizado para gestión de contenedores.

Es un proyecto de código abierto que automatiza el despliegue de aplicaciones dentro de contenedores de software, proporcionando una capa adicional de abstracción y automatización de virtualización de aplicaciones en múltiples sistemas operativos.

Su entorno de ejecución, Docker Engine, permite compilar y ejecutar contenedores en cualquier equipo de desarrollo y, después, almacenar o compartir imágenes de contenedor mediante un registro de contenedor, como Docker Hub o Azure Container Registry".



Información:

- Tipo de programa: software libre y de código abierto.
- Autor Solomon: Hykes.
- Desarrollador: Docker, Inc.
- Lanzamiento inicial: marzo de 2013.
- Licencia: Apache License 2.0.
- Programado en: Go.

Go es un lenguaje de programación concurrente y compilado inspirado en la sintaxis de C, que intenta ser dinámico como Python y con el rendimiento de C o C++. Ha sido desarrollado por Google y actualmente está disponible en formato binario para los sistemas operativos Windows, GNU/Linux, FreeBSD y Mac OS X, pudiendo también ser instalado en estos y en otros sistemas mediante el código fuente.

Go es un lenguaje de programación compilado, concurrente, imperativo, estructurado, orientado a objetos y con recolector de basura que de momento es soportado en diferentes tipos de sistemas UNIX, incluidos Linux, FreeBSD, Mac OS X y Plan 9 (puesto que parte del compilador está basado en un trabajo previo sobre el sistema operativo Inferno). Las arquitecturas soportadas son i386, amd64 y ARM.

Inferno es un sistema operativo inicialmente creado por Bell Labs, y actualmente desarrollado y mantenido por Vita Nuova Holdings. Fue diseñado con el objetivo de ser compacto, distribuido en red, dispositivos y plataformas, también posee muchas características avanzadas y coloca a disposición del usuario un gran conjunto de herramientas. Se puede obtener como Software Libre, en términos similares a GNU/Linux o BSD. El nombre del sistema y muchos de sus programas asociados, así como el nombre mismo de la compañía Vita Nuova, están inspirados en la obra literaria de Dante Alighieri, Divina comedia.

- Última versión estable (a fecha agosto de 2021): 20.10.8.



+ Info

La empresa 451 Research, a través de su unidad operativa Uptime Institute, que proporciona investigación para operadores de centros de datos, indica Docker como:

"Una herramienta que puede empaquetar una aplicación y sus dependencias en un contenedor virtual que se puede ejecutar en cualquier servidor Linux. Esto ayuda a permitir la flexibilidad y portabilidad en donde la aplicación se puede ejecutar, ya sea en las instalaciones físicas, la nube pública, nube privada, etc."



Docker utiliza características de aislamiento de recursos del kernel Linux, para permitir que "contenedores" independientes se ejecuten dentro de una sola instancia de Linux, evitando la sobrecarga de iniciar y mantener máquinas virtuales.

- El soporte del kernel Linux para los espacios de nombres aísla la vista que tiene una aplicación de su entorno operativo.

(Incluyendo árboles de proceso, red, ID de usuario y sistemas de archivos montados).

- Los cgroups del kernel proporcionan aislamiento de recursos.

(Incluyendo la CPU, la memoria, el bloque de E/S y de la red).

Desde la versión 0.9, Docker:

- Incluye la biblioteca libcontainer como su propia manera de utilizar directamente las facilidades de virtualización que ofrece el kernel Linux.
- Utiliza las interfaces abstraídas de virtualización mediante libvirt, LXC (Linux Containers) y systemd-nspawn.



+ Info

Docker se puede integrar con diferentes herramientas de infraestructura, como:

Amazon Web Services, Ansible, Cfengine, Chef, Google Cloud Platform, DigitalOcean, IBM Bluemix, Jelastic, Jenkins, Microsoft Azure, OpenStack Nova, OpenSVC, Puppet, Salt, y Vagrant.

Historia

Solomon Hykes comenzó Docker como un proyecto interno dentro dotCloud,³¹ empresa enfocado a una plataforma como un servicio (PaaS).

Docker representa una evolución de la tecnología patentada de dotCloud, que es a su vez construida sobre proyectos de código abierto anteriores como Cloudlets.

Veamos su evolución:

- **2013.**

En marzo de 2013 fue liberado como código abierto.



- **2014.**

En marzo de 2014, con el lanzamiento de la versión 0.9, Docker dejó de utilizar LXC como el entorno de ejecución por defecto y lo reemplazó con su propia biblioteca, libcontainer, escrito en Go.

- **2015.**

El 13 de abril de 2015, el proyecto tenía:

- Más de 20.700 estrellas de GitHub.
- Convirtiéndolo en uno de los proyectos con más estrellas de GitHub, (en 20ª posición).
- Más de 4 700 bifurcaciones (forks).
- Casi 900 colaboradores.

En ingeniería de software se considera una bifurcación (en inglés fork) al desarrollo de un proyecto informático tomando como base un código fuente que ya existe o a la ramificación de un proyecto madre en varios proyectos que son independientes entre sí y que cuentan con objetivos o desarrolladores diferentes. Como resultado de una bifurcación se pueden derivar varios proyectos de uno preexistente, los cuales pueden intentar cubrir necesidades distintas, aunque similares o implementar diferentes soluciones con el fin de abordar los mismos problemas pudiendo llegar a competir entre ellos.

- **2018.**

Un análisis mostró las siguientes organizaciones como las principales contribuyentes de Docker:

- Red Hat.
- mayores contribuyentes, más que el equipo de Docker en sí.
- el equipo de Docker.
- Microsoft.
- IBM.
- Google.
- Cisco Systems.
- Y Amadeus IT Group.



- **2020 (aparición de un malware).**

El 29 de julio de 2020 se dio a conocer la existencia de Doki, un malware que corre en el sistema operativo Linux que tiene por finalidad infectar la API de los contenedores Docker mal configurados.

Algunas de sus acciones son:

- Crea URL única con vidas cortas para descargar payloads durante el ataque.
- Ha sido creado para ejecutar comandos recibidos desde sus operadores.
- Usa la biblioteca TLS para funciones criptográficas.

9.1.1. Comandos principales

Vamos a ver algunos de los comandos más importantes en el uso de Docker:

- **docker.**

Sin argumentos, nos mostrará todos los comandos que tenemos disponibles.

- **docker -v.**

Muestra la versión que tenemos instalada.

Si deseamos obtener más información sobre la instalación, podemos usar el comando `docker version` o `docker info`.

- **docker run hello-world.**

Comprueba que la instalación funciona correctamente, ejecutamos un contenedor de prueba o «hello world».

Mostrará un mensaje de bienvenida en el caso de que el funcionamiento haya sido correcto.

La imagen de hello-world es un ejemplo, (con un único fichero `hello.c`) que imprime el mensaje en la terminal. El comando predeterminado de esta imagen es ejecutar este binario.

- **Comandos de Contenedor** (seguido del identificador del contenedor o de su nombre).

- Lifecycle (ciclo de vida de un contenedor).

- » **docker create.**

Crea un contenedor, pero no lo comienza.



» **docker run.**

Crea y comienza un contenedor en una operación.

» **docker rename.**

Permite renombrar al contenedor.

» **docker rm.**

Eliminar un contenedor. Podemos indicar una lista de identificadores separados por espacios para que realicen la acción en todos ellos.

» **docker update.**

Actualiza los recursos limitados de un contenedor.

- Comenzar y detener.

» **docker start.**

Iniciar contenedor.

» **docker stop.**

Parar un contenedor en ejecución. Podemos indicar una lista de identificadores separados por espacios para que realicen la acción en todos ellos.

» **docker restart.**

Detiene y comienza un contenedor.

» **docker pause.**

Pausa un contenedor corriendo, se puede decir que "lo congela".

» **docker unpause.**

Quita la pausa de un contenedor corriendo.

» **docker wait.**

Bloquea hasta que un contenedor corriendo se detiene.

» **docker kill.**

Envía una SIGKILL (mata) a un contenedor corriendo.



» **docker attach.**

Se conecta a un contenedor corriendo.

» **docker cleanup.**

Limpia todos los contenedores en funcionamiento.

- Información de contenedores.

» **docker ps.**

Muestra los contenedores corriendo.

» **docker logs.**

Obtiene logs de un container.

» **docker inspect.**

Observa toda la info en un contenedor.

» **docker events.**

Obtiene eventos de un contenedor.

» **docker port.**

Muestra el puerto público de un contenedor.

» **docker top.**

Muestra los procesos corriendo en un contenedor.

» **docker stats.**

Muestra las estadísticas de recursos usados por contenedor.

Con la opción `-all` (`docker stats -all`) muestra una lista de los contenedores corriendo.

» **docker diff.**

Muestra los archivos cambiados en el FS del contenedor.

» **docker ps.**

Listar los contenedores en ejecución en nuestro sistema

Si queremos listar también los contenedores que no se encuentran en ejecución (contenedores corriendo y detenidos) en ese momento lo podremos hacer añadiendo el argumento `-a` (**`docker ps -a`**).



+ Info

Volúmenes en Contenedores Docker.

Podemos agregar almacenamiento de datos externo, lo que hace posible crear directorios que se montarán en el contenedor cada vez que éste ejecute. Sobre este directorio podrán trabajar las aplicaciones.

Ejemplo que ejecuta un contenedor con el software httpd y tiene montado el directorio /tmp/testdir de la máquina host en la ruta /root/testdir del contenedor:

```
> mkdir /tmp/testdir
```

```
> docker run -v /tmp/testdir:/root/testdir httpd
```

- **Comandos de imagen de un contenedor.**

- **docker image --help.**

Muestra un listado de todos los comandos que puedes ejecutar para trabajar con imágenes.

- **docker pull.**

Descargar una imagen de un repositorio.

- **docker image ls.**

Listar las imágenes descargadas en el sistema.

Para que se muestren también las imágenes intermedias hay que utilizar la opción **-a** (docker image ls -a), y para ver los números de identificación, (ID) se indica la opción **-q**.

- **docker import.**

Crea una imagen de un tarball (archivo comprimido de una imagen).

- **docker build.**

Crea imagen de un Dockerfile.



Los Dockerfiles son archivos que se utilizan para crear imágenes Docker mediante programación, y que permiten crear de forma rápida y reproducible una imagen de Docker (utilizando el comando de `docker build`).

(Dockerfiles contiene instrucciones para construir una imagen de Docker, donde cada instrucción se escribe en una fila y se da en la forma).

- **docker commit.**

Crea imagen de un contenedor, pausándolo temporalmente si está corriendo.

- **docker rm.**

Elimina una o varias imágenes.

También se puede utilizar **docker rmi**. Puedes indicar el nombre de la imagen o parte del número de identificación de la imagen, no es necesario indicar todo el ID de la imagen. (Si existen dos imágenes y empiezan con diferentes caracteres, puedes utilizar solo un carácter).

No es posible borrar una imagen si existe un contenedor.

- **docker load.**

Carga una imagen de un archivo tar como STDIN, incluyendo imágenes y tags.

- **docker save.**

Salva una imagen a un archivo tar a STDOUT con todas las capas padre, tags y versiones.

- **docker history.**

Muestra el historial de una imagen.

- **docker tag.**

Taggea una imagen a un nombre asignado.

- **docker run -i -t image_name /bin/bash.**

Ejecutar imagen como contenedor.

- **docker exec -it /bin/bash.**

Para acceder a la terminal de un contenedor en ejecución de forma interactiva (dicho contenedor debe tener disponible bash).



- **Comandos equivalentes de imágenes.**
 - docker images es equivalente a docker image ls.
 - docker rmi es la versión corta docker image rm.
 - docker pull alternativa a docker image pull.
 - docker push es equivalente a docker image push.



+ Info

Puedes obtener más información en la web oficial de Docker.

<https://docs.docker.com/engine/reference/run/>

9.2. Azure Kubernetes Service

Azure Kubernetes Service (AKS) ofrece Kubernetes sin servidor, proporciona integración y entrega continuas (CI/CD) integrada y seguridad y gobernanza de nivel empresarial.

Ofrece una sola plataforma para crear, entregar y escalar aplicaciones.

AKS proporciona el aprovisionamiento de clústeres mediante la plataforma de Azure, la línea de comando de Azure o mediante herramientas de infraestructura como código como Azure Resources Manager y Terraform.

9.3. Kubernetes

Es la plataforma de orquestación de contenedores de código abierto desarrollada por Google y mantenida ahora por la Cloud Native Computing Foundation.

Es una poderosa herramienta para el despliegue automatización, escalamiento y administración de componentes.

Kubernetes se basa en un modelo que define los componentes básicos y los utiliza para gestionar las actividades relacionadas con el desarrollo de software.

Kubernetes funciona con Docker, y se ha incorporado a múltiples plataformas de nube.



La API de Kubernetes permite manejar programación básica de unidades llamadas pods, dentro de los cuales residen los contenedores, que se conectan con un volumen de almacenamiento (ya sea un directorio en el disco local o un disco en red).

Se simplifica la administración de contenedores y creación de servicios, los que son grupos de pods que trabajan en conjunto.

La combinación de un registro de contenedor con Kubernetes permite reforzar automáticamente la seguridad y calidad en los contenedores en todo su proceso de implementación.

Kubernetes proporciona una variedad de controles de seguridad y operacionales, como las políticas de seguridad de red y pod (recursos a nivel del clúster).

9.3.1. Objetos básicos de Kubernetes

- **Pods:** Un pod es la unidad más pequeña y básica en Kubernetes. Representa un único proceso en un contenedor y puede contener uno o varios contenedores. Los pods son escalables y proporcionan aislamiento y recursos compartidos a los contenedores dentro de ellos.
- **Servicios:** Los servicios definen conjuntos de pods y una política de acceso para acceder a ellos. Proporcionan una abstracción que permite a las aplicaciones descubrir y comunicarse con otros componentes de la aplicación sin conocer sus direcciones IP.
- **ReplicaSets:** Una ReplicaSet garantiza que un número especificado de réplicas de un pod se estén ejecutando en todo momento. Si un pod falla, la ReplicaSet crea uno nuevo para reemplazarlo.
- **Deployment:** Los objetos de implementación administran actualizaciones de aplicaciones, lo que permite el despliegue y escalado de aplicaciones de manera declarativa. Los Deployments se utilizan para definir el estado deseado de la aplicación y Kubernetes se encarga de llevar la aplicación a ese estado.
- **Namespace:** Los namespaces son espacios aislados lógicos que se utilizan para organizar y gestionar recursos en un clúster de Kubernetes. Permiten la segmentación y el aislamiento de recursos y aplicaciones en un clúster.
- **ConfigMap y Secret:** ConfigMap es un objeto que almacena datos de configuración, mientras que Secret almacena datos confidenciales, como contraseñas y tokens. Estos objetos permiten separar la configuración de la aplicación de los archivos de definición de pods.
- **Volume:** los Volumes permiten la persistencia aun si los Pods son destruidos o reiniciados, son objetos que definen almacenamientos persistentes para los Pods.
- **PersistentVolumes (PV) y PersistentVolumeClaims (PVC):** Estos objetos se utilizan para gestionar el almacenamiento persistente en clústeres de Kubernetes. Un PersistentVolume es una unidad de almacenamiento, y un PersistentVolumeClaim es una solicitud para el acceso a ese almacenamiento.



¡Atención!

En Kubernetes, los valores que permiten identificar unívocamente un objeto se encuentran en el campo metadata del objeto. En la Metadata encontramos subcampos como metadata.name, metadata.namespace que unidos proporcionan un identificador único dentro de un cluster, o metadata.uid que garantiza la unicidad en todo el cluster, incluso aunque cambien los subcampos name y namespace.

9.3.2. Objetos avanzados

- **StatefulSets:** Los StatefulSets son utilizados para aplicaciones que requieren identidad y persistencia de red. Permiten la gestión de aplicaciones con estados, como bases de datos, en un clúster de Kubernetes.
- **DaemonSets:** Los DaemonSets se utilizan para garantizar que un pod se ejecute en todos los nodos del clúster. Son útiles para tareas de infraestructura, como recolección de registros o monitoreo.
- **Job y CronJob:** Los objetos Job y CronJob se utilizan para ejecutar trabajos en el clúster. Los CronJobs permiten la programación de trabajos de acuerdo a un horario específico.

9.3.3. Principales protocolos utilizados en Kubernetes

HTTP/HTTPS:

Utilizado para la comunicación entre servicios, API y aplicaciones desplegadas. HTTPS es preferido para garantizar seguridad mediante cifrado TLS.

gRPC: Basado en HTTP/2, optimiza la comunicación entre microservicios con Protocol Buffers (protobuf), soportando autenticación y balanceo de carga.

TCP:

Facilita conexiones confiables entre servicios y bases de datos dentro del clúster, utilizado en Pods, Services y StatefulSets.



UDP:

Priorizado en aplicaciones de baja latencia como streaming y gaming. Kubernetes permite definir servicios UDP.

IP (IPv4/IPv6):

Asigna direcciones IP a cada Pod dentro del clúster, facilitando la integración con redes externas y balanceadores de carga.

ICMP:

Usado para verificar conectividad y diagnosticar redes dentro del clúster, útil en herramientas como ping.

VXLAN:

Protocolo de redes virtuales superpuestas empleado en CNI (Container Network Interface) como Flannel y Calico.

DNS:

Kubernetes implementa un servicio DNS interno con CoreDNS, proporcionando nombres resolubles a Pods y Servicios.

SCTP (Stream Control Transmission Protocol):

Protocolo de transporte orientado a conexión que admite múltiples flujos de datos en un solo enlace, mejorando la resiliencia en aplicaciones de telecomunicaciones y transmisión de datos en tiempo real.

FTP (File Transfer Protocol):

Protocolo utilizado para la transferencia de archivos dentro del clúster o hacia entornos externos, aunque es menos seguro sin cifrado.

SSH (Secure Shell):

Protocolo utilizado para acceso remoto seguro y gestión de nodos dentro del clúster, proporcionando autenticación y cifrado de datos.



9.4. Hyper-V Containers

Esta plataforma maneja virtualización anidada dentro de Hyper-V, permitiendo a los usuarios acceder a Docker, y también proporciona sus propios cmdlets de PowerShell desde la línea de comandos.

Hyper-V fue es una plataforma de configuración ligera que inserta o saca contenedores del Docker Hub o de repositorios locales.

Cada contenedor de Hyper-V contiene una copia del kernel de Windows y cuenta con memoria asignada, para generar un fuerte aislamiento, valioso para entregar aislamiento del tipo de una máquina virtual, así se pueden correr aplicaciones no firmadas y aplicaciones multitenant (tenencia múltiple) en la misma instancia.

Microsoft introdujo Hyper-V Containers con Windows Server 2016, al mismo tiempo que Windows Server Containers.

9.5. OpenShift

La plataforma de contenedores de OpenShift, es un producto bajo premisas y ofrecido como Plataforma como servicio, desarrollada por Red Hat.

Se apoya en contenedores de Docker que son orquestados por Kubernetes y funcionan sobre Red Hat.

El entorno a través de arquitecturas conectables y bajo demanda, maneja tanto aplicaciones heredadas como nativas de la nube.

9.6. Otras soluciones de Orquestación de Contenedores

Debido al auge cada vez mayor de desarrollar las aplicaciones en contenedores, y que puedan trasladarse a La Nube, se ha producido una creciente demanda de soluciones de software de **Orquestación de Contenedores**.

Vamos a indicar algunas herramientas y servicios de orquestación de contenedores disponibles en la actualidad:

- **Amazon Elastic Container Service (ECS).**

Se trata de un servicio alojado que proporciona Amazon Web Services (AWS) para ejecutar contenedores Docker a escala en su infraestructura.

- **Azure Container Instance (ACI).**

Es el servicio básico que proporciona Microsoft Azure de orquestación de contenedores.



- **Azure Service Fabric.**

Es el servicio de código abierto que proporciona Microsoft Azure como Orquestador de contenedores.

- **Maratón.**

Marco (framework) para ejecutar contenedores a escala en Apache Mesos.

- **Nomad.**

Orquestador de contenedores que proporciona HashiCorp.

- **Docker Swarm.**

Es parte de Docker Engine. Es el Orquestador de contenedores proporcionado por Docker, Inc.

10. Bibliografía

- CARPIO, J., MÍGUEZ, J. V., MARTÍNEZ, S., GUIRADO, R., DEL VALLE-INCLÁN BOLAÑO, J. L. *Instalación y mantenimiento de sistemas informáticos*. Editorial Universidad Nacional de Educación a Distancia (UNED).
- <https://infosegur.wordpress.com/category/1-conceptos-basicos-de-la-seguridad-informatica/>.
- http://catarina.udlap.mx/u_dl_a/tales/documentos/lis/jerez_l_ca/capitulo1.pdf.
- <http://www.bsiconsultores.cl/descargas/C.9%20ConceptosdeSeguridaddeLaInformacion.pdf>.
- <https://es.slideshare.net/vaceitunofist/analisisy-gestionderiesgos>.
- <http://es.wikipedia.org>.
- <http://es.wikipedia.org>.
- <http://www.seguridadinformatica.unlu.edu.ar/?q=node/12>.
- <https://www.welivesecurity.com/la-es/2015/04/01/que-es-declaracion-de-aplicabilidad-soa/>.
- <https://www.infospymware.com/articulos/que-son-los-malwares/>.
- <https://support.kaspersky.com/mx/614>.
- <https://myslide.es/documents/analisisygestionderiesgos.html>.
- <https://www.optical.pe/tipos-de-ataques-informaticos-y-previsiones-para-el-2018/>.



- <https://sdei.unican.es/Paginas/informacion/infraestructura/CPDs.aspx>.
- <https://integriaims.com/mejores-software-de-help-desk/>.
- <https://www.capterra.com/help-desk-software/>.
- <https://es.slideshare.net/vaceitunofist/analisisy-gestionderiesgos>.
- <https://www.cloudhispano.com/que-son-los-data-center-tipos/>.
- <https://es.wikipedia.org/wiki/RJ-45>.
- https://es.wikipedia.org/wiki/Fibra_%C3%B3ptica_multimodo.
- <https://ciberseguridad.com/servicios/analisis-forense/>.
- <https://sii-concatel.com/microservicios-contenedores-y-kubernetes/>.
- <https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/los-microservicios-en-el-desarrollo-de-aplicaciones/>.
- <https://blog.techdata.com/ts/latam/antes-de-entender-contenedores-entendamos-microservicios>.
- <https://revistabyte.es/tema-de-portada-byte-ti/un-mundo-de-contenedores/>.
- <https://revistabyte.es/tema-de-portada-byte-ti/un-mundo-de-contenedores/>.
- <https://blog.nubity.com/las-6-mejores-soluciones-de-administracion-de-contenedores/>.
- <https://blog.techdata.com/ts/latam/antes-de-entender-contenedores-entendamos-microservicios>.
- <https://cloud.google.com/architecture/service-meshes-in-microservices-architecture?hl=es-419>.
- <https://www.nginx.com/blog/what-is-a-service-mesh/>.
- <https://programmerclick.com/article/6166562189/>.
- <https://blog.techdata.com/ts/latam/gu%C3%ADa-esencial-de-contenedores>.
- [https://es.wikipedia.org/wiki/Docker_\(software\)](https://es.wikipedia.org/wiki/Docker_(software)).
- [https://es.wikipedia.org/wiki/Go_\(lenguaje_de_programaci%C3%B3n\)](https://es.wikipedia.org/wiki/Go_(lenguaje_de_programaci%C3%B3n)).
- [https://es.wikipedia.org/wiki/Inferno_\(sistema_operativo\)](https://es.wikipedia.org/wiki/Inferno_(sistema_operativo)).
- <https://blog.techdata.com/ts/latam/orquestaci%C3%B3n-de-contenedores>.



- https://www.trendmicro.com/es_es/what-is/container-security.html.
- <https://hackernoon.com/service-mesh-with-envoy-101-e6b2131ee30b>.
- <https://www.redhat.com/es/topics/devops/what-is-blue-green-deployment>.
- <https://programmerclick.com/article/50831276543/>.